

2. SDS (Software Design Specification)

Magic Mint



[Revision history]

Revision date	Version #	Description	Author
11/01/2023	0.00	First Draft	남기원
11/30/2023	0.01	1차 수정	남기원

= Contents =

1. Introduction	01
2. Use case analysis	02
3. Class diagram	22
4. Sequence diagram	40
5. State machine diagram	61
6. User interface prototype	63
7. Implementation requirements	85
8. Glossary	86
9. References	87

1. Introduction

본 문서는 우리 팀에서 개발하고자 하는 게임인 'Magic Mint'의 Software Design Specification(SDS)이다. 본 문서에서는 Magic Mint를 위해 기존에 식별한 기능적 요구사항을 구현하기 위해 게임을 기능적 관점에서 바라보고 설계한다. Use case diagram은 사용자 관점에서 게임이 사용자에게 제공하는 기능을 나타내고, Class diagram은 게임 소프트웨어의 클래스 간 관계를 나타낸다. Sequence diagram과 State machine diagram은 게임 소프트웨어 시스템의 동적 관점을 묘사했다. User interface는 게임에서 사용자가 접하는 UI들에 관해 서술한다.

본 문서 작성 시 가장 중요하게 생각한 점은 Diagram 간의 일관성이다. 즉 여러 Diagram 간 모순되는 부분이 없도록 주의하여 작성했다. 또한 이전 단계에서 산출된 SRS 문서에 작성된 기능들이 빠짐없이 Diagram에 반영되어 작성되었는지를 확인하고 SRS 문서와 본 문서 간의 차이 또는 모순이 없도록 하였다.

Magic Mint는 게임 프로그램과 멀티 플레이를 위한 서버로 이루어져 있다. 게임 프로그램의 경우 유니티 엔진을 이용하여 개발하였고, 게임 서버의 경우 Photon 서버를 이용하여 개발하였다. 본 문서는 게임 기능적인 시스템 설계를 중심으로 각 Diagram을 다루었으며 상세 설명이 Description을 통해 서술되어 있다. 본 문서의 각 Diagram은 SRS 문서를 기반으로 사용자 요구사항 분석과 시스템 구현에 있어 시스템 구조를 쉽게 파악할 수 있도록 한다.

2. Use case analysis

이번 장은 Use case diagram과 Use case description을 제공한다. Diagram에 관한 고려 사항은 다음과 같다.

- Use case diagram은 description에 대한 부가적인 결과물에 불과하다. 따라서 본 문서를 읽는 독자는 Use case diagram보다 Use case description에서 Use cases에 대한 충분한 정보를 얻어야 한다.
- Use case diagram에 나타난 Use case는 모두 user-level use case이다.
- Player는 게임을 플레이하는 User이다.
- 본 프로젝트의 Use case는 총 18가지로, 각각의 Use case는 SRS의 다음과 같은 기능들을 포괄한다.
 - GameStart : 게임 시작하기
 - Dice : 주사위 굴리기
 - PlayerMovement : 이동하기
 - PlayerAttack : 공격하기, 스킬 사용하기, 적의 사망, 경험치 획득하기, 레벨업하기
 - PlayerRespawn : 플레이어의 사망, 게임 오버
 - NetworkLobby_Host : 방 생성하기, 서버 접속하기, 방 나가기
 - NetworkLobby_Client : 방 들어가기, 서버 접속하기, 방 나가기
 - GetItem_Drop : 장비 획득하기
 - GetItem_Shop : 상점 사용하기
 - Equipment : 장비 획득하기, 상점 사용하기
 - TurnEnd : 턴 종료하기, 적의 공격하기, 적의 이동하기
 - ChangeLanguage : 언어 변경 기능
 - Chatting : 채팅하기
 - Emoji : 상호작용
 - BlockUser : 채팅 차단하기
 - StatInfo : 플레이어 정보 확인하기, 적 정보 확인하기
 - MapFinder : 맵 탐색하기
 - GameClear : 게임 클리어

아래의 [그림 2-1]은 본 프로젝트의 Use case diagram이다.



[그림 2-1] - Use case diagram

이번 장의 남은 부분은 Use case description에 할당한다.

Use case #1 : PlayerMovement	
GENERAL CHARACTERISTICS	
Summary	플레이어가 맵을 이동하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어 맵 이동 기능을 사용하기 위해서는 플레이어가 게임을 플레이 중이어야 한다.
Trigger	플레이어가 맵을 이동하기 위해 이동 버튼을 눌렀을 경우
Success Post Condition	플레이어가 포인트를 소모하여 맵의 원하는 위치로 이동한다.
Failed Post Condition	플레이어가 이동할 수 없다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 맵 위를 이동한다.
1	이 Use case는 플레이어가 맵 이동을 위해 이동 버튼을 누를 때 시작된다.
2	플레이어는 이동이 가능한 범위 타일 가운데 이동할 타일을 선택한다.
3	선택한 타일로의 경로와 소모 포인트를 확인하고 이동할 타일을 다시 선택하면 포인트를 소모하여 플레이어가 이동한다.
4	이 Use case는 플레이어 이동이 완료되면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
1	1a. 이동에 필요한 포인트가 부족하여 이동할 수 없다. ...1a1. 플레이어가 이동이 가능한 타일이 표시되지 않는다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이에 평균 50번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #2 : PlayerAttack	
GENERAL CHARACTERISTICS	
Summary	플레이어가 적을 공격하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어 적 공격 기능을 사용하기 위해서는 플레이어가 게임을 플레이 중이어야 한다.
Trigger	플레이어가 적을 공격하기 위해 공격 버튼을 눌렀을 경우
Success Post Condition	플레이어가 포인트를 소모하여 공격 범위 내의 적을 공격한다.
Failed Post Condition	플레이어가 공격할 수 없다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 적을 공격한다.
1	이 Use case는 플레이어가 적 공격을 위해 공격 버튼을 누를 때 시작된다.
2	플레이어는 공격이 가능한 범위 내의 적 가운데 공격할 적을 선택한다.
3	플레이어의 공격력과 소모 포인트를 확인하고 포인트를 소모하여 플레이어 공격력만큼의 데미지를 적에게 준다.
4	이 Use case는 플레이어 공격이 완료되면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
1	1a. 스킬을 습득한 경우 스킬을 통한 공격을 사용할 수 있다. ...1a1. 스킬 습득 시 스킬 버튼이 활성화되어 선택할 수 있다. ...1a2. 스킬 사용 시나리오는 누르는 버튼에만 차이가 있을 뿐 이후의 시나리오는 공격과 동일하다.
3	3a. 공격에 필요한 포인트가 부족하여 공격할 수 없다. ...3a1. 플레이어 공격이 실행되지 않고 취소된다.
4	4a. 플레이어의 공격으로 인해 적의 체력이 전부 소모되면 적은 사망한다. ...4a1. 적이 사망하면 플레이어는 경험치를 획득할 수 있다. ...4a2. 플레이어의 경험치가 일정량을 넘어가면 레벨업을 한다. ...4a3. 적이 사망한 위치에는 일정 확률로 장비 아이템이 드롭된다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이에 평균 50번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #3 : Dice	
GENERAL CHARACTERISTICS	
Summary	플레이어가 주사위를 굴러 포인트를 획득하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 주사위를 굴러 포인트를 얻기 위해서는 플레이어가 게임을 플레이 중이어야 한다.
Trigger	플레이어가 게임을 시작하거나 적 턴에서 다시 플레이어 턴으로 넘어올 때
Success Post Condition	플레이어가 주사위를 굴러 이동, 공격 등 플레이어 행동에 필요한 포인트를 획득한다.
Failed Post Condition	-
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 주사위를 굴러 포인트를 획득한다.
1	이 Use case는 게임 시작 시와 플레이어 턴 시작 시 시작된다.
2	플레이어는 화면의 주사위를 클릭하여 주사위를 굴린다.
3	주사위를 굴려서 나온 수가 플레이어가 해당 턴에 사용할 수 있는 포인트가 된다.
4	이 Use case는 플레이어 포인트 정산이 완료되면 종료된다.
EXTENSION SCENARIOS	
Step	-
-	-
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이에 평균 50번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #4 : Equipment	
GENERAL CHARACTERISTICS	
Summary	플레이어가 장비를 착용하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 장비를 착용하기 위해서는 게임을 진행 중 장비를 획득해야 한다.
Trigger	플레이어가 게임 진행 중 장비 아이템을 획득했을 때
Success Post Condition	플레이어가 획득한 장비를 착용한다.
Failed Post Condition	플레이어가 획득한 장비를 착용하지 못했다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 획득한 장비를 착용한다.
1	이 Use case는 게임 진행 중 장비 아이템을 획득하면 시작된다.
2	플레이어가 획득한 장비가 장비 부위에 맞게 착용이 된다.
3	이 Use case는 플레이어 장비 착용이 완료되면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
2	2a. 플레이어가 획득한 장비의 장비 부위에 이미 장비가 있다. …2a1. 플레이어가 장비 교체 여부를 선택한다. …2a2. 플레이어가 장비 교체를 선택하면 기존의 장비를 버리고 획득한 장비를 착용한다. …2a3. 플레이어가 장비 교체를 선택하지 않는다면 기존의 장비를 유지하고 획득한 장비를 버린다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이에 평균 10번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #5 : GetItem_Drop	
GENERAL CHARACTERISTICS	
Summary	플레이어가 맵에 드롭된 아이템을 획득하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 드롭된 장비를 얻기 위해서는 플레이어가 게임을 플레이 중 적을 쓰러뜨려 적이 장비 아이템을 드롭해야 한다.
Trigger	플레이어가 장비가 드롭된 칸으로 이동했을 때
Success Post Condition	플레이어가 드롭된 장비를 획득하여 장착한다.
Failed Post Condition	플레이어가 드롭된 장비를 장착하지 않는다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 드롭된 아이템을 획득하여 장착한다.
1	이 Use case는 게임 중 플레이어가 장비 아이템이 드롭된 칸으로 이동했을 때 시작된다.
2	플레이어는 장비 아이템 정보를 확인한다.
3	플레이어는 장비 아이템 정보 창의 상호작용 버튼을 통해 획득한 장비를 착용하게 된다.
4	이 Use case는 플레이어가 획득한 장비를 착용하면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
3	3a. 플레이어가 획득한 장비의 장비 부위에 이미 장비가 있다. ...3a1. 플레이어가 장비 교체 여부를 선택한다. ...3a2. 플레이어가 장비 교체를 선택하면 기존의 장비를 버리고 획득한 장비를 착용한다. ...3a3. 플레이어가 장비 교체를 선택하지 않는다면 기존의 장비를 유지하고 획득한 장비를 버린다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이에 평균 10번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #6 : GetItem_Shop	
GENERAL CHARACTERISTICS	
Summary	플레이어가 상점을 통해 아이템을 획득하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 상점에서 장비를 얻기 위해서는 플레이어가 게임 플레이 중 상점으로 이동해야 한다.
Trigger	플레이어가 상점 칸으로 이동했을 때
Success Post Condition	플레이어가 상점에서 장비를 획득하여 장착한다.
Failed Post Condition	플레이어가 장비를 획득하지 않는다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 상점에서 아이템을 획득하여 장착한다.
1	이 Use case는 게임 중 플레이어가 상점 칸으로 이동했을 때 시작된다.
2	플레이어는 상점에서 판매 중인 아이템 리스트와 아이템 정보를 확인한다.
3	플레이어는 아이템 리스트에서 원하는 장비를 선택하고 포인트를 지불하고 장비를 획득한다.
4	플레이어는 획득한 장비를 착용한다.
5	이 Use case는 플레이어가 획득한 장비를 착용하면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
3	3a. 플레이어가 보유한 포인트가 상점 장비의 요구 포인트보다 적다. ...3a1. 플레이어는 보유 포인트가 부족하여 장비를 획득할 수 없다.
4	4a. 플레이어가 획득한 장비의 장비 부위에 이미 장비가 있다. ...4a1. 플레이어가 장비 교체 여부를 선택한다. ...4a2. 플레이어가 장비 교체를 선택하면 기존의 장비를 버리고 획득한 장비를 착용한다. ...4a3. 플레이어가 장비 교체를 선택하지 않는다면 기존의 장비를 유지하고 획득한 장비를 버린다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이에 평균 10번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #7 : PlayerRespawn	
GENERAL CHARACTERISTICS	
Summary	플레이어가 사망 시 부활하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 부활하기 위해서는 게임을 진행 중 사망해야 한다.
Trigger	플레이어가 게임 진행 중 사망했을 때
Success Post Condition	플레이어가 부활한다.
Failed Post Condition	플레이어가 부활하지 못하고 게임 오버된다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 사망 시 부활한다.
1	이 Use case는 게임 진행 중 사망했을 경우 시작된다.
2	플레이어가 보유한 목숨이 있다면 목숨을 하나 소모해서 부활한다.
3	이 Use case는 플레이어가 부활하면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
2	2a. 플레이어가 보유한 목숨이 없다. ...2a1. 플레이어는 부활하지 못하고 게임 오버가 된다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이에 평균 3번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #8 : TurnEnd	
GENERAL CHARACTERISTICS	
Summary	플레이어가 턴을 종료하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 턴을 종료하기 위해서는 게임 진행 중 플레이어 턴이어야 한다.
Trigger	플레이어가 턴을 마치기 위해 턴 종료 버튼을 눌렀을 때
Success Post Condition	플레이어가 턴을 마친 후 적 턴을 진행한 후 적 턴이 종료된다.
Failed Post Condition	-
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 턴을 종료하고 적 턴이 진행되고 적 턴이 종료된다.
1	이 Use case는 플레이어가 턴 종료 버튼을 눌렀을 때 시작된다.
2	플레이어가 턴 종료 버튼을 누르면 플레이어 턴이 종료되고 적 턴이 시작된다.
3	적 턴이 시작되면 적은 플레이어를 공격한 후 턴을 종료한다.
4	이 Use case는 플레이어 턴이 종료되고 적 턴까지 종료되면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
3	3a. 적 공격 범위에 플레이어가 없다. ...3a1. 적은 플레이어를 인식하지 못하고 아무 행동도 하지 않은 채 턴을 종료한다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이에 평균 50번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #9 : NetworkLobby_Host	
GENERAL CHARACTERISTICS	
Summary	플레이어가 방을 만드는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 방을 만들기 위해서는 게임 플레이 전 게임 시스템에 접속하여야 한다.
Trigger	플레이어가 게임 플레이를 위해 방 만들기 버튼을 눌렀을 경우
Success Post Condition	플레이어가 방을 만들어 방에 들어온다.
Failed Post Condition	-
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 방을 만든다.
1	이 Use case는 플레이어가 게임 로비에서 Create Room 버튼을 누르면 시작된다.
2	플레이어가 방을 만들면 다른 플레이어들은 게임 로비에서 생성된 방 리스트에서 해당 방을 확인할 수 있게 된다.
3	이 Use case는 플레이어가 방을 만들고 만들어진 방에 들어오면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
1	1a. 플레이어가 방을 만들기 전 게임 시스템에 접속해야 한다. ...1a1. 플레이어가 닉네임을 입력하고 게임 접속을 한다. ...1a2. 플레이어가 닉네임을 입력하지 않았다면 알림창을 표시한다. ...1a3. 게임 시스템 접속이 완료되면 게임 로비로 들어와 해당 기능을 사용할 수 있다. 1b. 플레이어가 방을 만들 때 이름을 입력할 수 있다. ...1b1. 플레이어가 입력한 이름의 방이 만들어진다. ...1b2. 플레이어가 이름을 입력하지 않았다면 'Room40'과 같이 임의의 방 이름이 'Room+번호'로 주어진다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이에 게임 전 평균 1번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #10 : NetworkLobby_Client	
GENERAL CHARACTERISTICS	
Summary	플레이어가 방에 참가하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 방에 참가하기 위해서는 게임 플레이 전 게임 시스템에 접속하여야 하고 다른 플레이어가 만든 방이 있어야 한다.
Trigger	플레이어가 게임 플레이를 위해 방 리스트에서 방에 참가한 경우
Success Post Condition	플레이어가 선택한 방에 들어온다.
Failed Post Condition	플레이어가 선택한 방에 들어갈 수 없다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 방에 들어간다.
1	이 Use case는 플레이어가 방 리스트에서 원하는 방을 선택하면 시작된다.
2	플레이어가 방을 선택하면 해당 방에 들어가 다른 플레이어와 같이 게임할 수 있다.
3	이 Use case는 플레이어가 선택한 방에 들어오면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
1	1a. 플레이어가 방을 만들기 전 게임 시스템에 접속해야 한다. ...1a1. 플레이어가 닉네임을 입력하고 게임 접속을 한다. ...1a2. 플레이어가 닉네임을 입력하지 않았다면 알림창을 표시한다. ...1a3. 게임 시스템 접속이 완료되면 게임 로비로 들어와 해당 기능을 사용할 수 있다. 1b. 플레이어가 선택한 방의 정원이 전부 찼다. ...1b1. 플레이어는 선택한 방에 들어갈 수 없다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이에 게임 전 평균 1번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #11 : Change Language	
GENERAL CHARACTERISTICS	
Summary	플레이어가 게임 시스템의 언어를 바꾸는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 게임 언어를 변경하기 위해서는 게임 시스템에 접속한 상태여야 한다.
Trigger	플레이어가 언어 변경 버튼을 누른 경우
Success Post Condition	게임 UI에서 제공되는 언어가 변경된다.
Failed Post Condition	게임 UI에서 제공되는 언어가 변경되지 않는다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 게임 UI의 언어를 변경한다.
1	이 Use case는 플레이어가 언어 변경 버튼을 누르면 시작된다.
2	플레이어가 원하는 언어의 언어 변경 버튼을 누르면 선택한 언어로 UI의 언어가 변경된다.
3	이 Use case는 게임 UI의 언어가 모두 변경되면 종료된다.
EXTENSION SCENARIOS	
Step	-
-	-
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이 중 평균 1번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #12 : GameStart	
GENERAL CHARACTERISTICS	
Summary	플레이어가 게임을 시작하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 게임을 시작하기 위해서는 게임 방에 들어온 상태여야 한다.
Trigger	플레이어가 게임 시작 버튼을 누른 경우
Success Post Condition	게임이 시작된다.
Failed Post Condition	게임이 시작되지 않는다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 게임을 시작한다.
1	이 Use case는 플레이어가 게임 시작 버튼을 누르면 시작된다.
2	플레이어가 게임 시작 버튼을 누르면 게임 화면이 로딩되며 맵을 불러온다.
3	맵 불러오기가 완료되면 적 배치 및 플레이어의 위치가 결정된다.
4	이 Use case는 게임 플레이 화면 로드가 완료되면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
1	1a. 게임 방에 여러 명의 플레이어가 있는 경우 …1a1. 게임 방에 참가한 플레이어는 준비 버튼을 눌러 준비 표시를 띄워 다른 플레이어가 준비 버튼을 누를 때까지 대기한다. …1a2. 게임 방의 모든 플레이어가 준비 버튼을 눌러 준비 상태가 되었을 때 게임 방을 만든 방장 플레이어가 시작 버튼을 누르면 게임이 시작된다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이 전 평균 1번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #13 : Chatting	
GENERAL CHARACTERISTICS	
Summary	플레이어 간 채팅 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 채팅을 하기 위해서는 게임 방에 들어온 상태여야 한다.
Trigger	플레이어가 채팅 창에 내용을 입력하고 보낸 경우
Success Post Condition	플레이어가 보낸 채팅이 다른 플레이어의 채팅 창에 표시된다.
Failed Post Condition	플레이어가 보낸 채팅이 다른 플레이어의 채팅 창에 표시되지 않는다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어 채팅 창을 통해 소통한다.
1	이 Use case는 플레이어가 채팅 창에 내용을 입력하고 보내면 시작된다.
2	플레이어가 채팅을 보내면 다른 플레이어 채팅 창에 보낸 내용이 표시된다.
3	이 Use case는 플레이어가 보낸 채팅이 다른 플레이어 채팅 창에 표시되면 종료된다.
EXTENSION SCENARIOS	
Step	-
-	-
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이 중 평균 1번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #14 : Emoji	
GENERAL CHARACTERISTICS	
Summary	플레이어 간 상호작용 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 상호작용을 하기 위해서는 게임 방에 들어온 상태여야 한다.
Trigger	플레이어가 상호작용을 위해 이모티콘을 보낸 경우
Success Post Condition	플레이어가 보낸 이모티콘이 이모티콘 창에 표시되어 다른 플레이어가 볼 수 있다.
Failed Post Condition	플레이어가 보낸 이모티콘이 이모티콘 창에 표시되지 않아 다른 플레이어가 볼 수 없다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 이모티콘을 활용하여 소통한다.
1	이 Use case는 플레이어가 이모티콘을 선택하면 시작된다.
2	플레이어가 이모티콘을 선택하면 해당 플레이어의 이모티콘 창에 선택한 이모티콘이 표시되고 다른 플레이어가 선택한 이모티콘을 볼 수 있다.
3	이 Use case는 플레이어가 보낸 이모티콘이 이모티콘 창에 표시되어 다른 플레이어가 이모티콘을 보면 종료된다.
EXTENSION SCENARIOS	
Step	-
-	-
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이 중 평균 20번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #15 : BlockUser	
GENERAL CHARACTERISTICS	
Summary	채팅 및 상호작용 차단 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 채팅 및 상호작용 차단을 위해서는 게임 방에 들어온 상태여야 한다.
Trigger	플레이어가 채팅 및 상호작용 차단을 위해 명령어를 입력했을 때
Success Post Condition	차단한 플레이어의 채팅 및 이모티콘이 채팅 창에 표시되지 않는다.
Failed Post Condition	차단한 플레이어의 채팅 및 이모티콘이 채팅 창에 표시된다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 다른 플레이어의 채팅 및 상호작용을 차단한다.
1	이 Use case는 플레이어가 채팅 및 상호작용 차단을 위해 명령어를 입력하면 시작된다.
2	차단하고 싶은 플레이어에 대해 채팅을 보낸 플레이어를 '/block (플레이어명)'을 통해 차단한다.
3	이 Use case는 차단한 플레이어가 보낸 채팅 및 이모티콘이 표시되지 않으면 종료된다.
EXTENSION SCENARIOS	
Step	-
-	-
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이 중 평균 20번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #16 : StatInfo	
GENERAL CHARACTERISTICS	
Summary	캐릭터 스탯을 확인하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 스탯을 확인하기 위해서는 게임 플레이 중인 상태이어야 한다.
Trigger	플레이어가 스탯 확인을 위해 플레이어 또는 적 캐릭터를 선택할 때
Success Post Condition	플레이어가 선택한 캐릭터의 스탯이 표시된다.
Failed Post Condition	플레이어가 선택한 캐릭터의 스탯이 표시되지 않는다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 캐릭터의 스탯을 확인한다.
1	이 Use case는 플레이어가 캐릭터를 선택하면 시작된다.
2	플레이어가 캐릭터를 선택하면 선택한 캐릭터의 스탯이 표시된다.
3	이 Use case는 플레이어가 선택한 캐릭터의 스탯이 표시되면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
1	1a. 플레이어가 플레이어 본인 캐릭터의 스탯을 확인한다. ...1a1. 플레이어가 본인 캐릭터의 스탯을 확인하기 위해서는 플레이어 본인 캐릭터의 플레이어 정보 버튼을 누른다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이 중 평균 10번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #17 : MapFinder	
GENERAL CHARACTERISTICS	
Summary	플레이어가 맵을 탐색하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 맵을 탐색하기 위해서는 게임을 플레이 중이어야 한다.
Trigger	플레이어가 맵을 탐색하기 위해 키보드나 마우스 조작을 가했을 때
Success Post Condition	플레이어가 원하는 방향으로 게임 화면이 이동하며 맵을 탐색한다.
Failed Post Condition	게임 화면이 이동하지 않는다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 맵을 탐색한다.
1	이 Use case는 플레이어가 맵 탐색을 위해 키보드나 마우스 조작을 가할 때 시작된다.
2	플레이어가 원하는 방향으로 게임 화면이 이동한다.
3	이 Use case는 플레이어가 맵 탐색을 종료하기 위해 스페이스 바를 누르면 종료된다.
EXTENSION SCENARIOS	
Step	Branching Action
1	1a. 플레이어가 키보드 조작을 사용한다. ...1a1. W를 누르면 화면이 위쪽으로, A를 누르면 왼쪽으로, S를 누르면 아래쪽으로, D를 누르면 오른쪽으로 이동한다. 1b. 플레이어가 마우스 조작을 사용한다. ...1b1. 마우스를 원하는 방향의 화면 가장자리로 이동시키면 이동시킨 방향으로 게임 화면이 이동한다.
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이 중 평균 10번
<Concurrency>	제한 없음
Due Date	2023.11.30.

Use case #18 : GameClear	
GENERAL CHARACTERISTICS	
Summary	플레이어가 게임의 목표를 달성해 클리어하는 기능
Scope	Game System
Level	User level
Author	남기원
Last Update	2023. 10. 31.
Status	Analysis
Primary Actor	게임 플레이어
Preconditions	플레이어가 게임을 클리어하기 위해서는 게임을 플레이 중이어야 한다.
Trigger	플레이어가 보스 몬스터를 처치했을 때
Success Post Condition	플레이어가 게임을 클리어하는데 성공한다.
Failed Post Condition	플레이어가 게임을 클리어하는데 실패한다.
MAIN SUCCESS SCENARIO	
Step	Action
S	플레이어가 게임을 클리어한다.
1	이 Use case는 플레이어가 보스 몬스터를 처치하면 시작된다.
2	플레이어가 보스 몬스터를 처치하면 게임이 클리어가 된다.
3	이 Use case는 게임이 클리어가 되면 종료된다.
EXTENSION SCENARIOS	
Step	-
-	-
RELATED INFORMATION	
Performance	≤ 2 seconds
Frequency	플레이어 당 게임 플레이 중 1번
<Concurrency>	제한 없음
Due Date	2023.11.30.

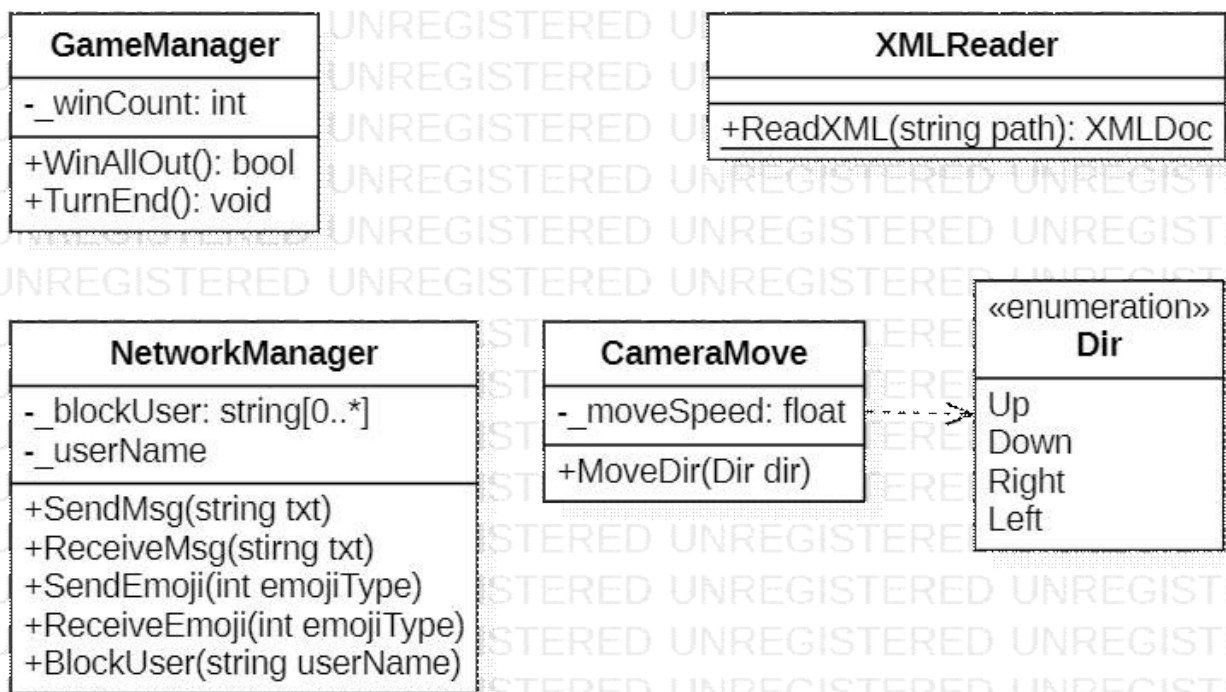
3. Class diagram

이번 장은 다양한 관점에서 바라본 Class diagram과 각각에 대한 설명을 기술한다. 본 문서의 Class diagram을 볼 때 고려해야 할 사항은 다음과 같다.

- 함수는 대문자로 이름을 시작한다.(PascalCase)
- 변수의 이름은 소문자로 시작하며 단어의 구분은 대문자로 한다.(camelCase)
- private 변수, 함수는 local 변수와 구분하기 쉽게 변수 이름 앞에 언더바를 붙인다.
- 기본적으로 멤버 변수는 private, 멤버 함수는 public으로 한다.
- private 함수는 Class Diagram에 그리지 않는다.

3.1. Main Class diagram

게임 진행에 있어 게임 운영을 관리하는 GameManager 클래스와 XML 파일 파싱을 위한 XMLReader 클래스, 네트워크 채팅 관리를 위한 NetworkManager 클래스, 게임의 시점이 되는 카메라를 관리하기 위한 Cam 클래스이다. 아래 [그림 3-1]은 Main Class diagram이다.



[그림 3-1] - Main Class diagram

GameManager			
Class			
Description	게임 진행에 있어 게임의 운영을 관리하는 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_winCount	int	private
	게임의 클리어를 위한 보스 몬스터 소환에 필요한 적 처치 수를 카운트하기 위한 변수		
구분	Name	Type	Visibility
	Description		
Operations	WinAllOut	bool	public
	적 처치 수가 보스 몬스터 소환 조건을 충족하는지 확인하기 위한 함수		
	TurnEnd	void	public
	플레이어의 턴을 종료하는 함수		

XMLReader			
Class			
Description	XML 파일을 파싱하기 위해 사용하는 클래스		
구분	Name	Type	Visibility
	Description		
Operations	ReadXML	XMLDoc	public
	XML 파일의 경로를 받아 해당 경로에 있는 XML 파일을 읽어 오는 함수		

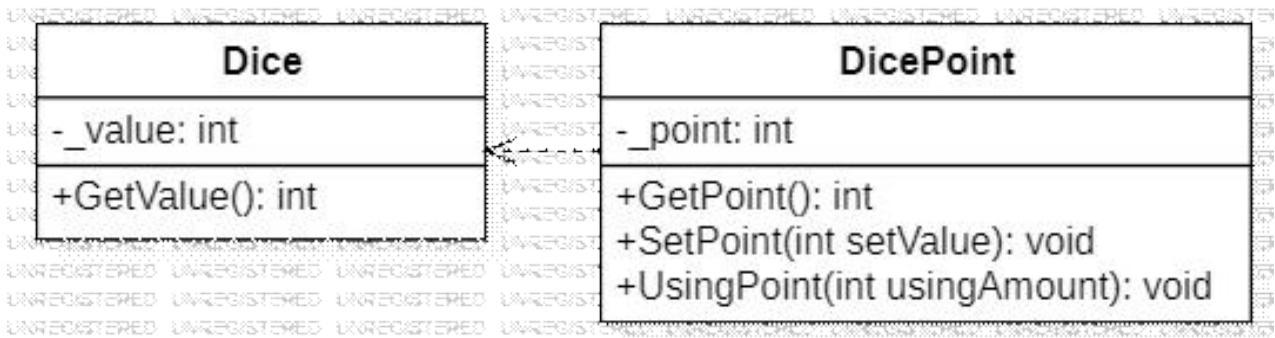
NetworkManager			
Class			
Description	플레이어 간 채팅을 관리하는 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_blockUser	int	private
	차단한 사람을 관리하기 위한 변수		
구분	Name	Type	Visibility
	Description		
Operations	SendMsg	bool	public
	채팅을 보내는 함수		
	ReceiveMsg		
	다른 플레이어가 보낸 채팅을 받는 함수		
	SendEmoji		
	상호작용을 보내는 함수		
	ReceiveEmoji		
	다른 플레이어가 보낸 상호작용을 받는 함수		
	BlockUser		
	특정 플레이어의 채팅을 차단하는 함수		

CameraMove			
Class			
Description	게임의 화면을 위한 카메라의 움직임을 관리하는 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_moveSpeed	float	private
	카메라의 이동 속도를 설정하기 위한 변수		
구분	Name	Type	Visibility
	Description		
Operations	MoveDir	void	public
	카메라의 이동 방향에 따라 카메라를 이동시키는 함수		

Dir			
Class			
Description	카메라 이동 방향을 관리하기 위한 열거형		
구분	Name	Type	Visibility
	Description		
Attribute	Up	-	-
	위 방향		
	Down	-	-
	아래 방향		
	Right	-	-
	오른쪽 방향		
	Left	-	-
	왼쪽 방향		

3.2. Dice System Class diagram

아래 [그림 3-2]는 플레이어 턴 시작 시 주사위를 굴리기 위한 Dice 시스템과 관련된 클래스들을 표현한 Dice System Class diagram이다.



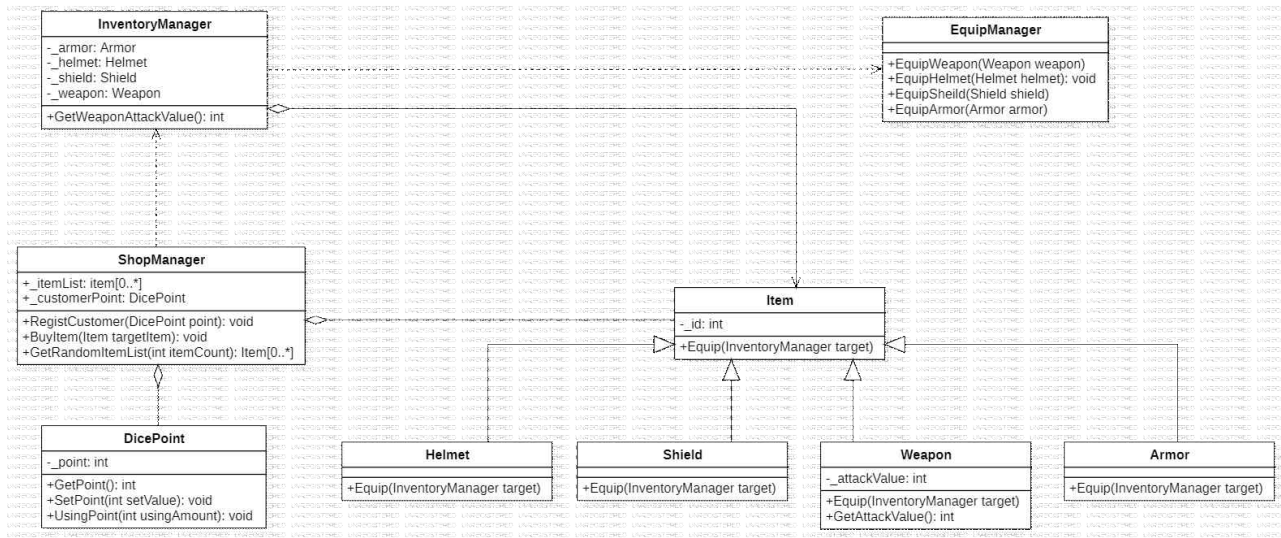
[그림 3-2] - Dice System Class diagram

Dice			
Class	주사위를 굴리고 포인트를 얻는 매커니즘을 관리하는 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_value	int	private
	주사위를 굴려서 얻은 포인트를 저장하기 위한 변수		
구분	Name	Type	Visibility
	Description		
Operations	GetValue	int	public
	주사위를 굴려 랜덤하게 값이 결정되는 매커니즘을 구현한 함수		

DicePoint			
Class	플레이어 포인트를 관리하는 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_point	DicePoint	private
	포인트를 나타내는 변수		
구분	Name	Type	Visibility
	Description		
Operations	GetPoint	int	public
	현재 포인트를 반환하는 함수		
	SetPoint	void	public
	포인트를 인자로 넘겨받은 포인트로 재설정하는 함수		
	UsingPoint	void	public
	포인트를 인자로 넘겨받은 포인트만큼 소모하는 함수		

3.3. Equipment Class diagram

아래 [그림 3-3]은 장비 시스템과 관련된 클래스들을 표현한 Equip Class diagram이다. DicePoint 클래스의 경우 3.2. 절의 DicePoint 클래스와 동일하므로 본 3.3. 절에서는 설명하지 않는다.



[그림 3-3] - Equipment Class diagram

ShopManager			
Class			
Description	상점을 관리하는 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_itemList	item	public
	상점에서 판매하는 아이템 목록을 나타내는 리스트 타입 변수		
	_customerPoint	DicePoint	public
구분	Name	Type	Visibility
	Description		
Operations	RegistCustomerItemList	void	public
	플레이어가 상점을 이용할 때 플레이어의 포인트와 상점 기능을 사용 가능하도록 GUI를 표시하는 함수		
	BuyItem	void	public
	플레이어가 아이템 구매 시 플레이어의 포인트를 가격만큼 소모하고 아이템을 플레이어에게 부여하는 함수		
	GetRandomItemList	List	public
	전체 아이템 리스트에서 상점에서 판매할 아이템 목록을 랜덤하게 생성하는 함수		

InventoryManager			
Class	플레이어 장비를 관리하는 클래스		
Description			
구분	Name	Type	Visibility
	Description		
Attribute	_armor	Armor	private
	플레이어의 갑옷		
	_helmet	Helmet	private
	플레이어의 투구		
	_shield	Shield	private
	플레이어의 방패		
	_weapon	Weapon	private
	플레이어의 무기		
구분	Name	Type	Visibility
	Description		
Operations	GetWeaponAttackValue	int	public
	플레이어 장비의 공격력을 전부 합산해서 반환하는 함수		

EquipManager			
Class	플레이어 장비 착용을 관리하는 클래스		
Description			
구분	Name	Type	Visibility
	Description		
Operations	EquipArmor	void	public
	플레이어에 갑옷을 장착		
	EquipHelmet	void	public
	플레이어에 투구를 장착		
	EquipShield	void	public
	플레이어에 방패를 장착		
	EquipWeapon	void	public
	플레이어에 무기를 장착		

Item			
Class	아이템 정보를 관리하는 클래스		
Description			
구분	Name	Type	Visibility
	Description		
Attribute	_id	int	private
	아이템 구분을 위해 부여한 개별 ID 코드 변수		
구분	Name	Type	Visibility
	Description		
Operations	Equip	void	public
	아이템을 장착하는 기능을 위한 함수		

Helmet			
Class Description	아이템 중 투구 부분의 아이템을 관리하는 클래스		
구분	Name	Type	Visibility
	Description		
Operations	Equip		public
	투구 아이템을 장착하는 기능을 위한 함수		

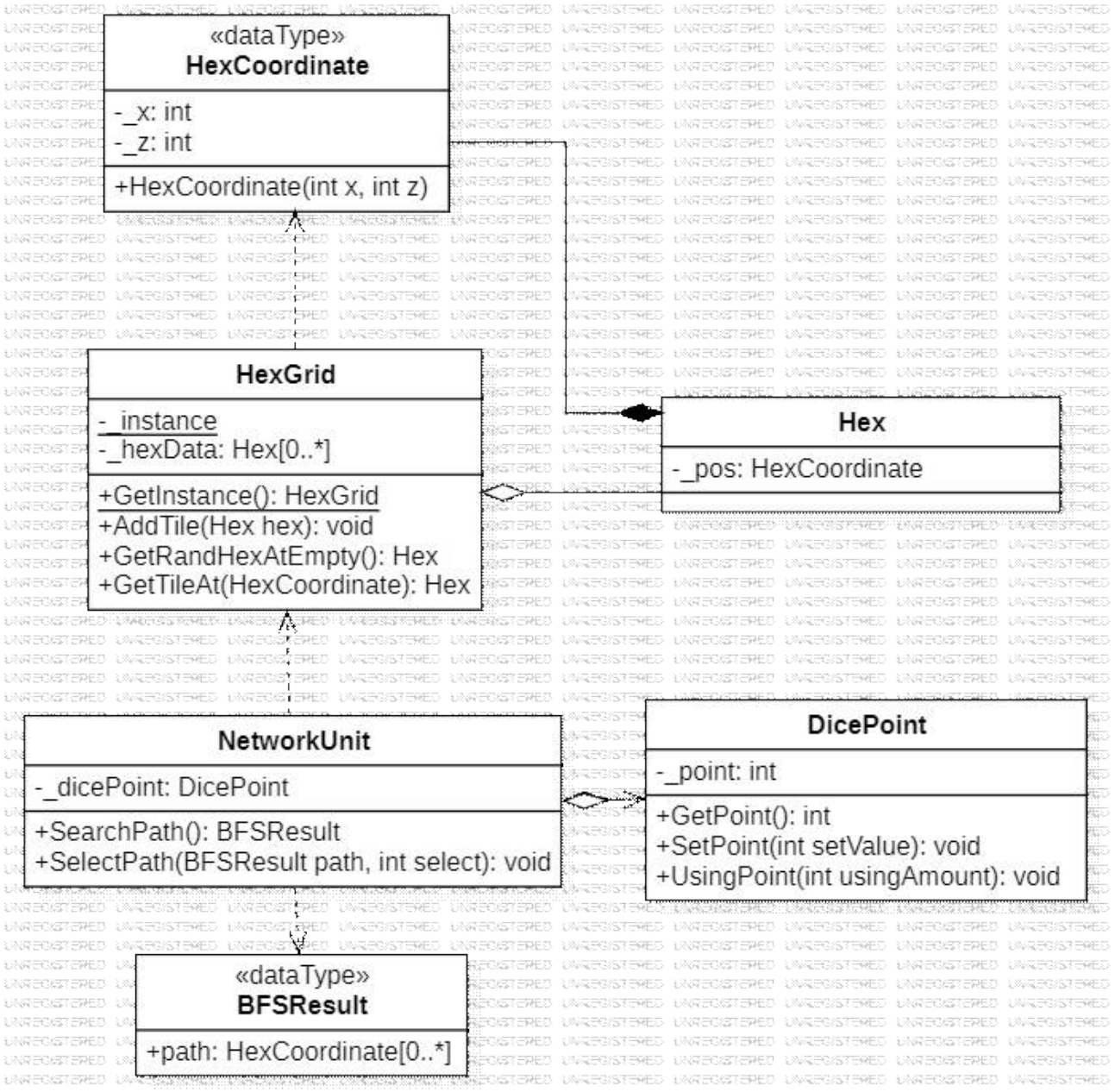
Shield			
Class Description	아이템 중 방패 부분의 아이템을 관리하는 클래스		
구분	Name	Type	Visibility
	Description		
Operations	Equip		public
	방패 아이템을 장착하는 기능을 위한 함수		

Weapon			
Class Description	아이템 중 무기 부분의 아이템을 관리하는 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_attack	int	private
	무기의 공격력을 저장하는 변수		
구분	Name	Type	Visibility
	Description		
Operations	Equip		public
	아이템을 장착하는 기능을 위한 함수		
	GetAttackValue	int	public
	무기의 공격력을 반환하는 함수		

Armor			
Class Description	아이템 중 갑옷 부분의 아이템을 관리하는 클래스		
구분	Name	Type	Visibility
	Description		
Operations	Equip		public
	갑옷 아이템을 장착하는 기능을 위한 함수		

3.4. Field Class diagram

아래 [그림 3-4]은 게임 플레이의 무대가 되는 게임 필드에 관련된 클래스들을 표현한 Field Class diagram이다. DicePoint 클래스의 경우 3.2. 절의 DicePoint 클래스와 동일하므로 본 3.4. 절에서는 따로 설명하지 않는다.



[그림 3-4] - Field Class diagram

Hex			
Class Description	각각의 타일을 관리하기 위한 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_pos	HexCoordinate	public
	타일의 위치 정보를 저장하는 변수		

HexCoordinate			
Class Description	HexGrid 상에서 위치를 표현하기 위한 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_x	int	public
	타일의 x 좌표		
	_z	int	public
	타일의 z 좌표		
구분	Name	Type	Visibility
	Description		
Operations	HexCoordinate		public
	타일의 위치 정보를 생성하는 생성자		

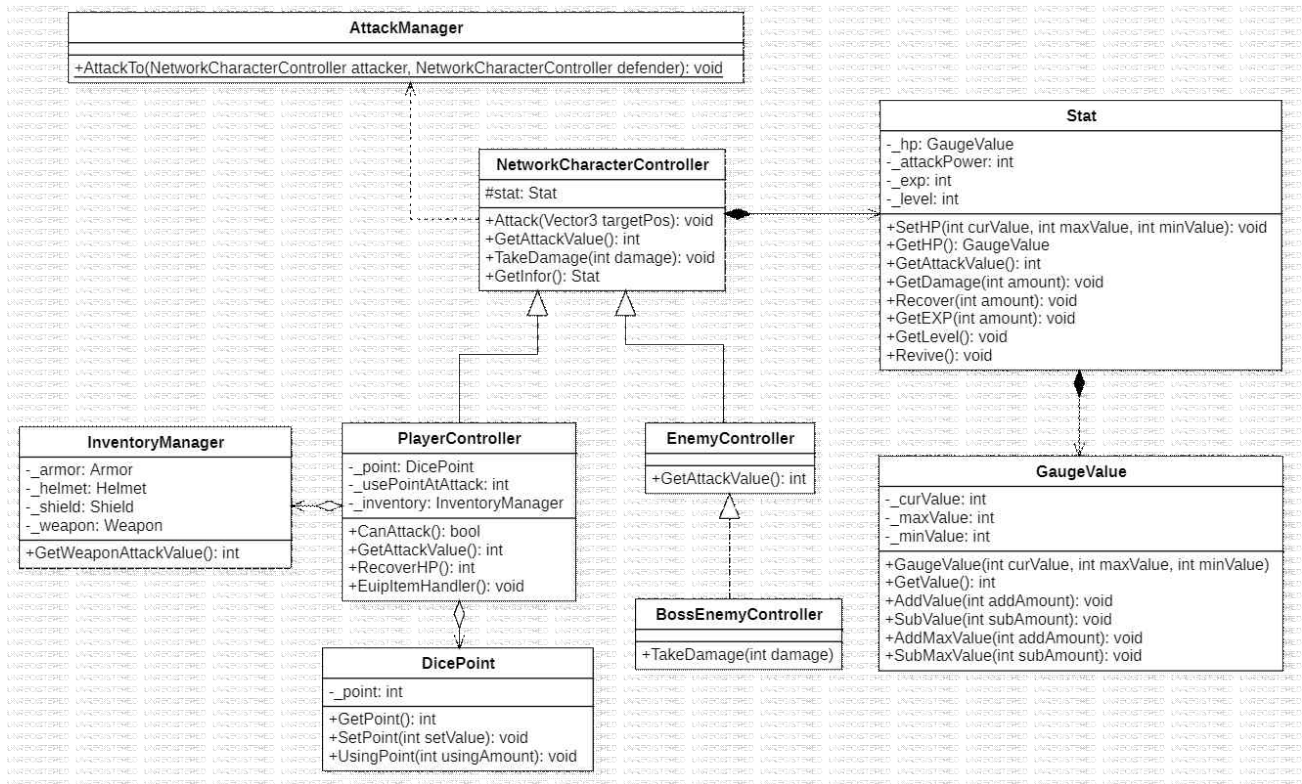
HexGrid			
Class Description	타일 간의 위치 정보를 계산하고 타일 위치 정보와 타일 타입을 매핑하기 위한 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_instance		private
	HexGrid를 싱글톤으로 관리하기 위한 변수		
	_hexData	Hex	private
	타일에 대한 정보를 저장하는 변수		
구분	Name	Type	Visibility
	Description		
Operations	GetInstance	HexGrid	public
	싱글톤으로 생성된 HexGrid를 반환하는 함수		
	AddTile	void	public
	좌표에 새로운 Hex를 추가하는 함수		
	GetRandHexAtEmpty	Hex	public
	현재 좌표에 등록된 Hex 중 빈 타일을 찾아 그 중 무작위로 반환하는 함수		
	GetTileAt	Hex	public
	인자로 주어진 위치에 존재하는 Hex를 반환하는 함수		
	GetNeighboursFor	Hex[0..*]	public
	인자로 주어진 타일의 주변 타일들의 Hex를 가져오는 함수		

NetworkUnit			
Class	현재 플레이어의 위치 타일과 이동을 위해 선택한 타일 간의 경로와 같은 경로 정보를 관리하기 위한 함수		
구분	Name	Type	Visibility
	Description		
Attribute	_dicePoint	DicePoint	private
	현재 플레이어가 가진 포인트를 저장한 함수		
구분	Name	Type	Visibility
	Description		
Operations	SearchPath	BFSResult	public
	플레이어의 타일 이동을 위해 경로를 탐색하는 함수		
	SelectPath	void	public
	결정한 이동 경로를 통해 실제 이동을 수행하는 함수		
	MoveToRandomNeighbor	void	public
	포인트 소모 없이 주변 타일로 랜덤하게 이동하는 함수		

BFSResult			
Class	플레이어가 이동하기 위한 경로 상의 타일들을 저장하기 위한 데이터 타입 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	path	HexCoordinate[0..*]	public
	플레이어가 이동하기 위한 경로상의 타일들이 저장된 리스트 타입의 변수		

3.5. Character Class diagram

아래 [그림 3-5]는 게임의 캐릭터가 사용하는 클래스들을 표현한 Character Class diagram이다. DicePoint 클래스의 경우 3.2. 절에서 설명한 DicePoint 클래스와 동일하고, InventoryManager 클래스의 경우 3.3. 절에서 설명한 InventoryManager 클래스와 동일, NetworkUnit 클래스의 경우 3.4. 절에서 설명한 NetworkUnit 클래스와 동일하므로 본 3.5. 절에서는 설명하지 않는다.



[그림 3-5] - Character Class diagram

PlayerController			
Class	플레이어 조작을 관리하는 클래스		
Description			
구분	Name	Type	Visibility
	Description		
Attribute	_point	DicePoint	private
	플레이어 행동에 필요한 포인트를 관리하기 위한 변수		
	_usePointAtAttack	int	private
	플레이어 공격에 소모되는 포인트를 저장하는 변수		
	inventory	InventoryManager	private
	플레이어 장비 수치를 반영하기 위한 InventoryManager 클래스를 사용하기 위한 변수		
구분	Name	Type	Visibility
	Description		
Operations	CanAttack	bool	public
	플레이어가 가지고 있는 현재 포인트와 공격에 소모되는 포인트를 비교하여 플레이어가 현재 공격 가능한지를 확인하는 함수		
	GetAttackValue	int	public
	플레이어 기본 공격력과 함께 장비 착용 유무를 판단하여 플레이어 총 공격력을 구하는 함수		
	RecoverHP	int	public
	플레이어 체력 회복 시스템을 구현한 함수		

BossEnemyController			
Class	보스 몬스터의 사망 여부를 통해 게임 클리어 조건을 관리하는 클래스		
Description			
구분	Name	Type	Visibility
	Description		
Operations	TakeDamage	void	public
	보스 몬스터가 TakeDamage 결과 사망 상태가 되면 게임 클리어 판정을 하는 함수		

NetworkCharacterController			
Class	게임 네트워크에서 플레이어 및 적 간의 행동 및 상호작용을 관리하는 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	stat	Stat	protected
	캐릭터의 스탯을 관리하는 변수		
구분	Name	Type	Visibility
	Description		
Operations	Attack	int	public
	인자로 주어진 캐릭터를 공격하는 함수		
	GetAttackValue	void	public
	캐릭터의 공격력을 반환하는 함수		
	TakeDamage	void	public
	캐릭터에게 인자로 받은 양만큼의 데미지를 입히는 함수		
	GetCharacterInfor	stat	public
	캐릭터의 stat을 받아 받은 stat을 플레이어에게 전달		

EnemyController			
Class	적 동작을 관리하기 위한 클래스		
구분	Name	Type	Visibility
	Description		
Operations	GetAttackValue	int	public
	적의 공격력을 반환하는 함수		
	AttackAction	void	public
	적 턴에 적의 이동 및 공격을 수행하는 함수		

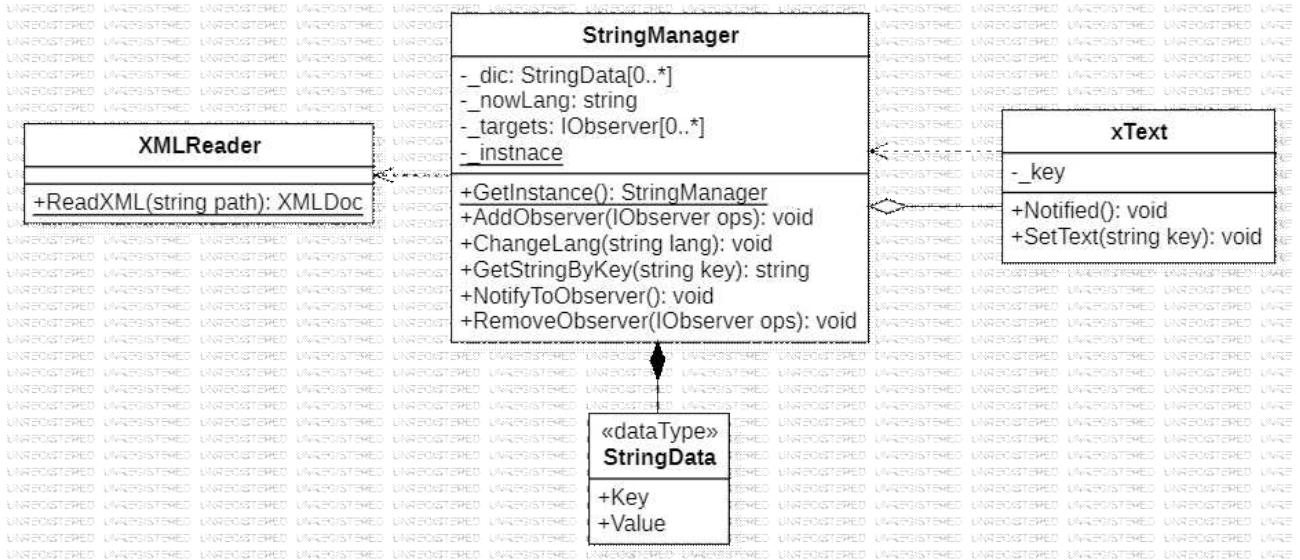
AttackManager			
Class	플레이어와 적 간의 공격 매커니즘을 관리하기 위한 클래스		
구분	Name	Type	Visibility
	Description		
Operations	AttackTo	void	public
	인자로 받은 attacker의 공격력만큼 defender에게 데미지를 가하는 함수		

Stat			
Class	캐릭터의 스탯을 관리하는 클래스		
Description			
구분	Name	Type	Visibility
	Description		
Attribute	_hp	GaugeValue	private
	캐릭터의 체력		
	_attackPower	int	private
	캐릭터의 공격력		
	_exp	int	private
	캐릭터의 경험치		
	_level	int	private
	캐릭터의 레벨		
구분	Name	Type	Visibility
	Description		
Operations	SetHP	int	public
	HP 최대치 및 최소치, 현재량을 설정하는 함수		
	GetHP	void	public
	현재 HP량을 반환하는 함수		
	GetAttackValue	void	public
	플레이어의 공격력을 반환하는 함수		
	GetDamage	void	public
	현재 HP에서 데미지로 입은 피해량만큼 소모하는 함수		
	Recover	void	public
	현재 HP에 회복받은 만큼 더하는 함수		
	GetExp	void	public
	현재 경험치량에 획득한 경험치만큼 더하는 함수		
	GetLevel	void	public
	캐릭터의 현재 레벨을 올리는 함수		
	Revive	void	public
	캐릭터를 부활시키는 함수		

GaugeValue			
Class			
Description	최대와 최소가 정해진 변수를 효과적으로 관리하기 위한 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_curValue	int	private
	현재 수치		
	_maxValue	int	private
	최대 수치		
	_minValue	int	private
	최소 수치		
구분	Name	Type	Visibility
	Description		
Operations	GaugeValue		public
	게이지의 최솟값, 최댓값, 현재값을 지정하는 생성자		
	GetValue	int	public
	현재값을 반환하는 함수		
	AddValue	void	public
	현재값을 증가시키는 함수		
	SubValue	void	public
	현재값을 감소시키는 함수		
	AddMaxValue	void	public
	최댓값을 증가시키는 함수		
	SubMaxValue	void	public
	최댓값을 감소시키는 함수		

3.6. Lang Class diagram

아래 [그림 3-6]은 게임 내 언어 변경 기능을 구현하기 위해 언어와 관련된 클래스들을 표현한 Lang Class diagram이다. XMLReader 클래스의 경우 3.1. 절에서 설명했으므로 본 3.6. 절에서는 설명하지 않는다.



[그림 3-6] - Lang Class diagram

StringManager			
Class			
Description	언어 변환 기능 중추를 담당하는 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_dic	StringData	private
	언어 변환을 위해 기준 언어와 변환할 언어 간의 딕셔너리 형태의 데이터		
	_nowLang	string	private
	현재 언어 타입		
	_target	IObserver	private
	언어를 바꿀 때 그에 영향을 받는 Observer를 저장하는 변수		
	_instance	int	private
	StringManager를 싱글톤으로 관리하기 위한 변수		
구분	Name	Type	Visibility
	Description		
Operations	GetInstance	StringManager	public
	싱글톤으로 생성된 HexGrid를 반환하는 함수		
	AddObserver	void	public
	언어 변경 시 영향을 받는 Observer를 저장하는 함수		
	ChangeLang	void	public
	언어를 바꾸는 함수		
	GetStringByKey	void	public
	키를 통해 해당하는 의미의 문자열을 반환하는 함수		
	NotifyToObserver	void	public
	언어가 변경되었을 때 Observer에 전달하는 함수		
	RemoveObserver	void	public
	언어 변경 시 영향을 받는 Observer를 삭제하는 함수		

xText			
Class			
Description	언어 변환을 위해 텍스트와 옵저버를 관리하기 위한 클래스		
구분	Name	Type	Visibility
	Description		
Attribute	_key		private
	언어의 의미를 저장하는 변수		
구분	Name	Type	Visibility
	Description		
Operations	Notified	void	public
	언어 변경 시 이를 전달받아 실행되는 함수		
	SetText	void	public
	해당 텍스트의 언어를 변환하는 함수		

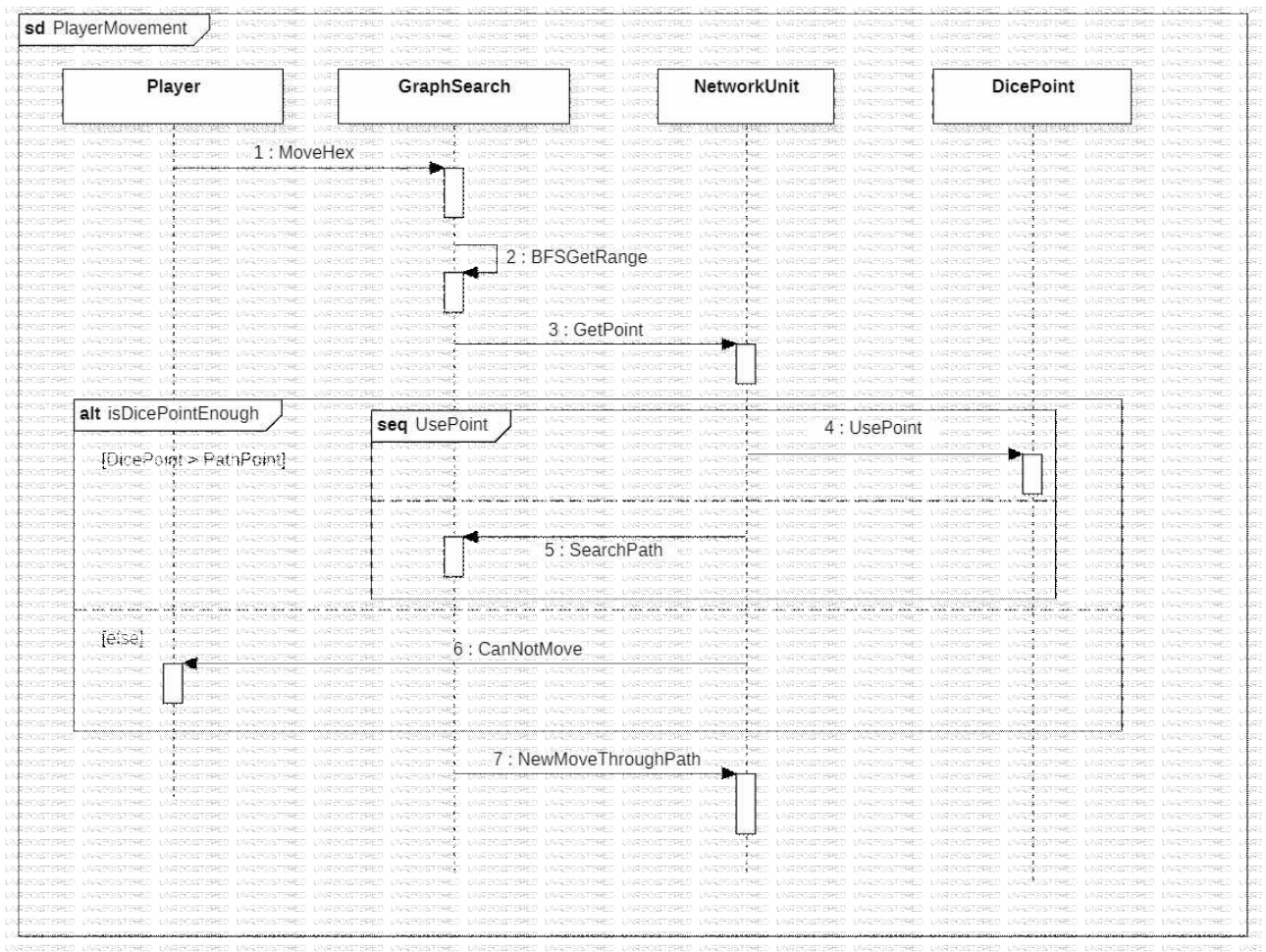
StringData			
Class	언어 변환을 위해 의미-언어 간의 데이터 타입을 관리하는 클래스		
Description	스		
구분	Name	Type	Visibility
	Description		
Attribute	Key		public
	언어의 의미 변수		
	Value		public
	실제 그 의미를 나타내는 언어의 텍스트		

4. Sequence diagram

이 장은 Sequence diagram과 그 설명을 제공한다. 고려 사항은 다음과 같다.

- 모든 Sequence diagram은 특정 Use case와 대응된다.
- 모든 Sequence diagram은 특정 Use case의 Main / Extension success scenario의 흐름을 따른다.
- 메시지를 주고받는 클래스들은 모두 class diagram에서 연결 관계가 있다.
- 흐름 묘사는 Sequence diagram를 그대로 글에 옮겨 적은 것으로, 일치성과 모순성을 검사하기 위해 사용됨과 동시에 diagram으로 이해되지 않는 부분을 해소하는데 사용한다.
- Player는 실제 코드 상에 있는 클래스가 아니라 사용자를 뜻하는 개념적 클래스로, Player와 상호작용하는 메시지의 경우 함수 또는 변수가 아닌 플레이어 행위 자체를 뜻한다.

4.1. PlayerMovement

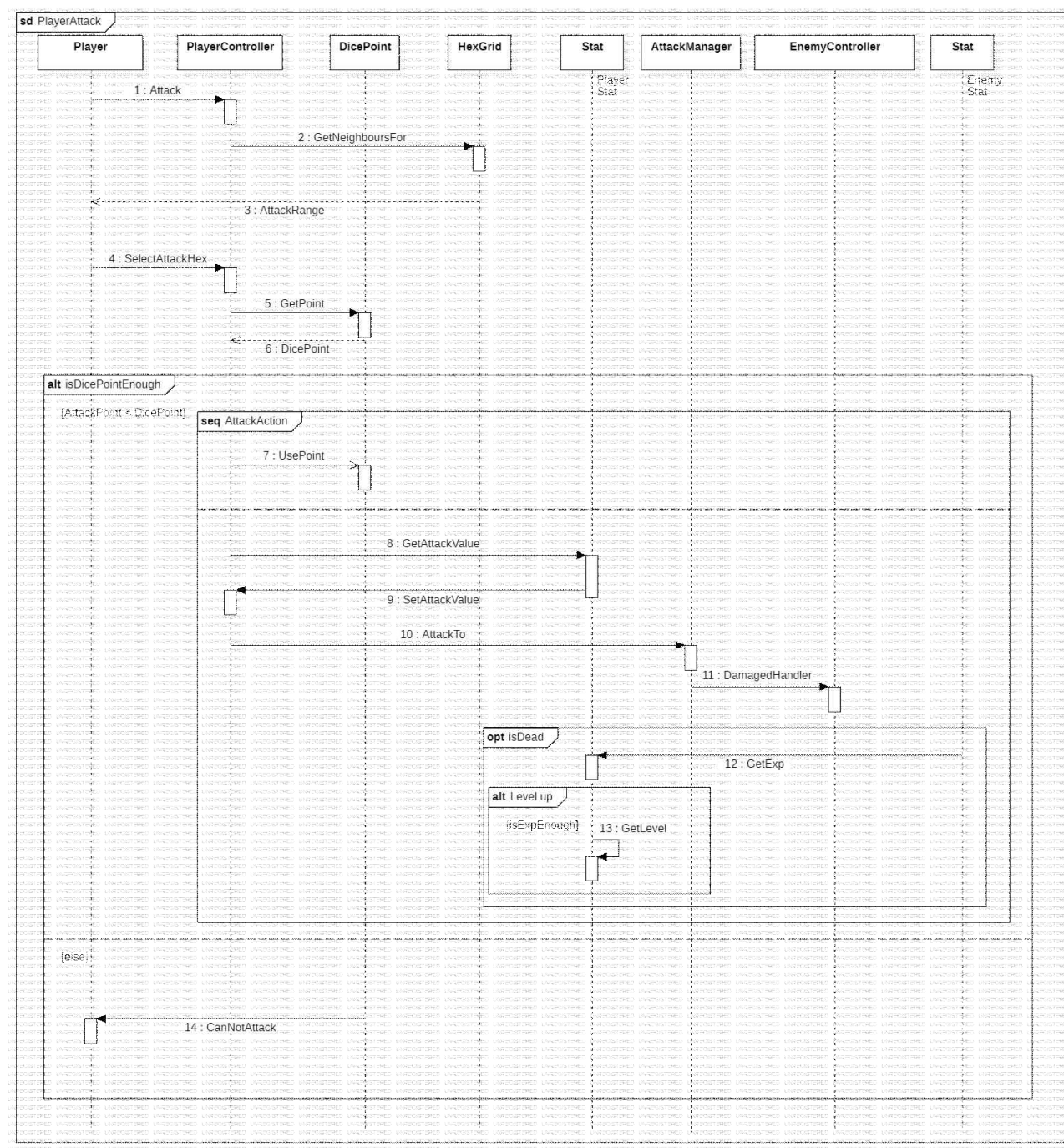


[그림 4-1] - PlayerMovement

위 [그림 4-1]은 사용자가 플레이어를 움직이는 Use case를 나타내는 Player Movement Sequence diagram이다.

우선 Player가 이동을 위해 타일을 선택하는 MoveHex 신호를 보내면 GraphSearch BFSGetRange를 통해 내부적으로 이동 범위를 산정한다. GraphSearch는 NetworkUnit 클래스의 SearchPath 함수를 명시적으로 설명하기 위해 SD에서 사용한 개념이다. GraphSearch에서 NetworkUnit에 GetPoint로 이동에 필요한 포인트를 요구하면 NetworkUnit에서는 포인트가 충분한지 판단한다. 만약 포인트가 충분하다면 DicePoint 클래스에서 이동에 필요한 포인트만큼 UsePoint를 통해 포인트를 사용하고 SearchPath 함수를 통해 GraphSearch에서 길을 찾도록하고, 포인트가 부족하다면 Player에게 움직일 수 없다는 CanNotMove 신호를 보내고 종료한다. 이동이 성립된 경우 GraphSearch는 NewMoveThroughPath 신호를 통해 NetworkUnit에 찾은 경로를 알려 플레이어 캐릭터를 이동시킨다.

4.2. PlayerAttack



[그림 4-2] - PlayerAttack

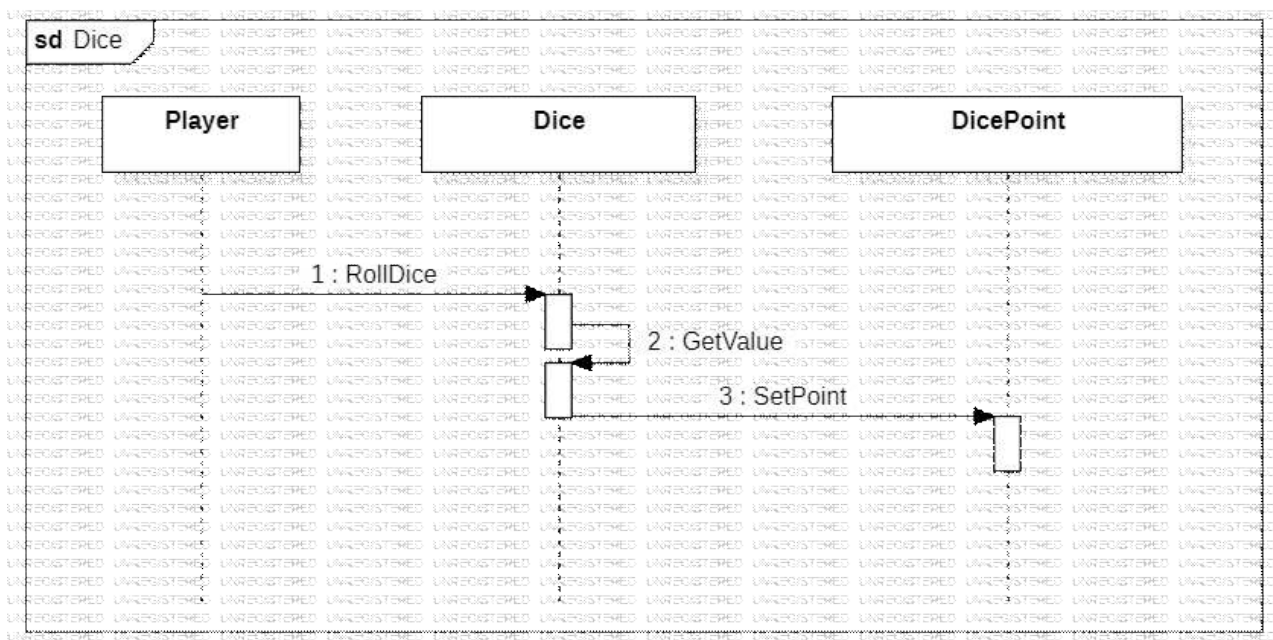
위 [그림 4-2]는 사용자가 플레이어를 통해 적을 공격하는 Use case를 나타내는 Sequence diagram이다.

Player가 적을 공격하기 위해 Attack 신호를 보내면 PlayerController에서는 GetNeighboursFor 신호를 HexGrid에 보내 AttackRange, 공격 범위에 대한 정보를

가져온다. 그 후 PlayerController가 Player에게 SelectAttackHex 신호를 통해 플레이어가 공격할 칸을 선택하면 PlayerController는 DicePoint에서 GetPoint 함수를 통해 공격에 필요한 Dice Point를 가져온다. 만약 포인트가 충분하지 않다면 플레이어에게 공격할 수 없다는 신호를 보내고, 포인트가 충분하다면 PlayerController는 공격에 필요한 포인트만큼을 DicePoint에게 UsePoint 함수를 실행하여 포인트를 소모하고 Stat 클래스의 GetAttackValue 함수를 통해 공격력을 산정한다. 그 후 AttackManager 클래스의 AttackTo 메소드를 통해 공격을 실행하면 AttackManager 클래스는 EnemyController 클래스의 DamageHandler를 통해 데미지를 산정한다.

데미지 산정 이후 적이 죽은 상태가 되면 적은 Stat 클래스의 GetExp 함수를 통해 플레이어에게 경험치를 주게 되고, 누적된 경험치가 일정량이 되면 자동으로 GetLevel 함수를 통해 레벨업이 진행된다.

4.3. Dice

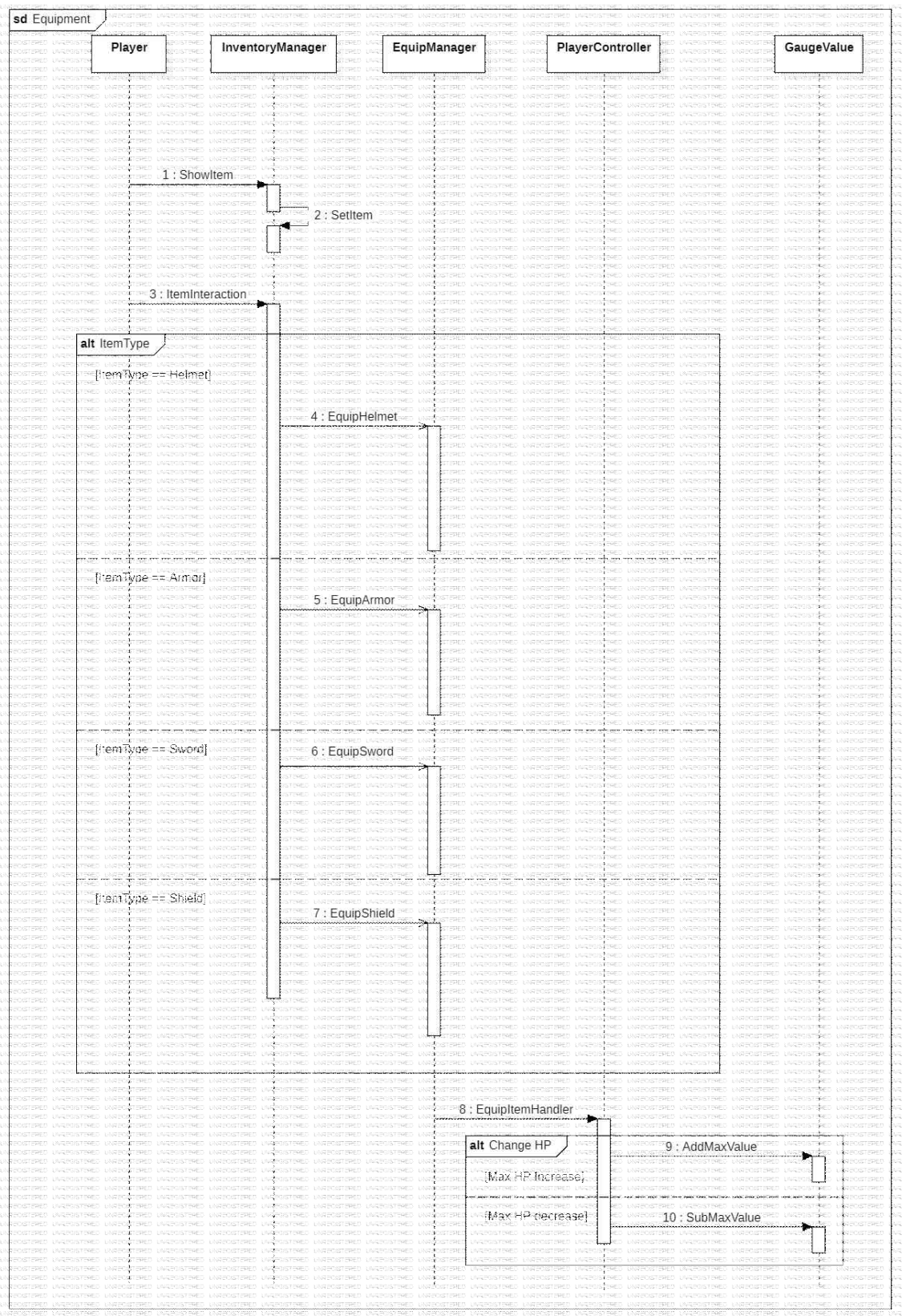


[그림 4-3] - Dice

위 [그림 4-3]은 사용자가 주사위를 굴러 포인트를 획득하는 Use case를 나타내는 Sequence diagram이다.

Player가 주사위를 굴리면 RollDice 신호를 통해 Dice 클래스에선 GetValue 함수를 통해 포인트를 산정하고 SetPoint 함수를 통해 DicePoint 클래스에 획득한 포인트를 저장한다.

4.4. Equipment

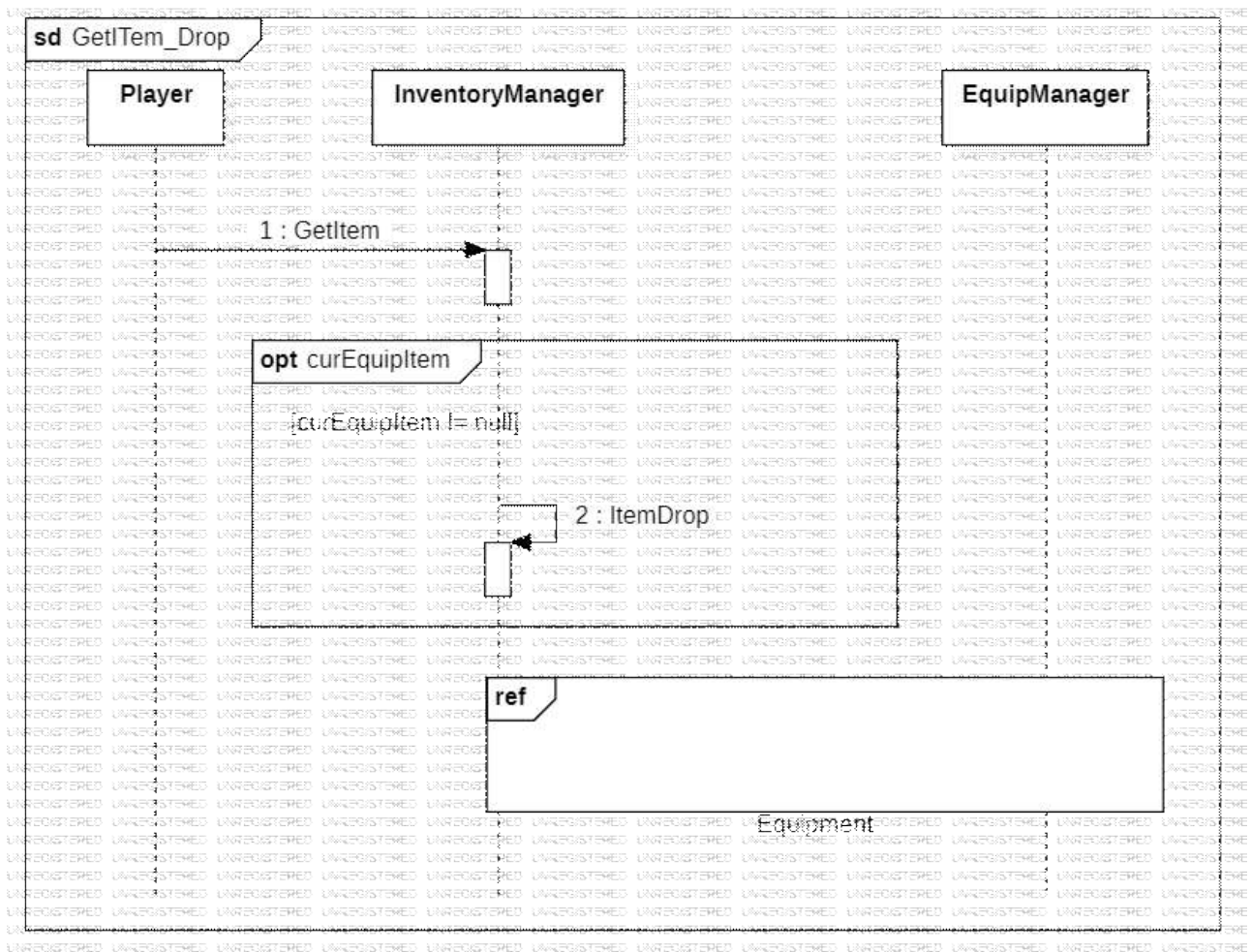


[그림 4-4] - Equipment

[그림 4-4]는 사용자가 장비를 착용하는 Use case를 나타내는 Sequence diagram이다.

Player가 장비 착용을 위한 ShowItem 신호를 보내면 InventoryManager에서는 내부적으로 SetItem 신호를 통해 아이템 정보를 가져오고 Player가 InteractionItem 신호를 보내면 해당 ItemType에 따라 Helmet, Armor, Weapon, Shield 별로 각각의 Equip 함수를 적용하여 아이템 착용을 한다. 아이템 착용이 완료된 시점에서 EquipManager에서는 EquipItemHandler 함수를 통해 PlayerController에서 장비 착용으로 인해 플레이어의 최대 HP의 변동 사항이 있는지를 체크해서 최대 HP가 증가한 경우에는 AddMaxValue 함수를, 최대 HP가 줄어든 경우에는 SubMaxValue 함수를 통해 GaugeVaule에서 Player의 HP 최대값 변동 사항을 반영한다.

4.5. GetItem_Drop



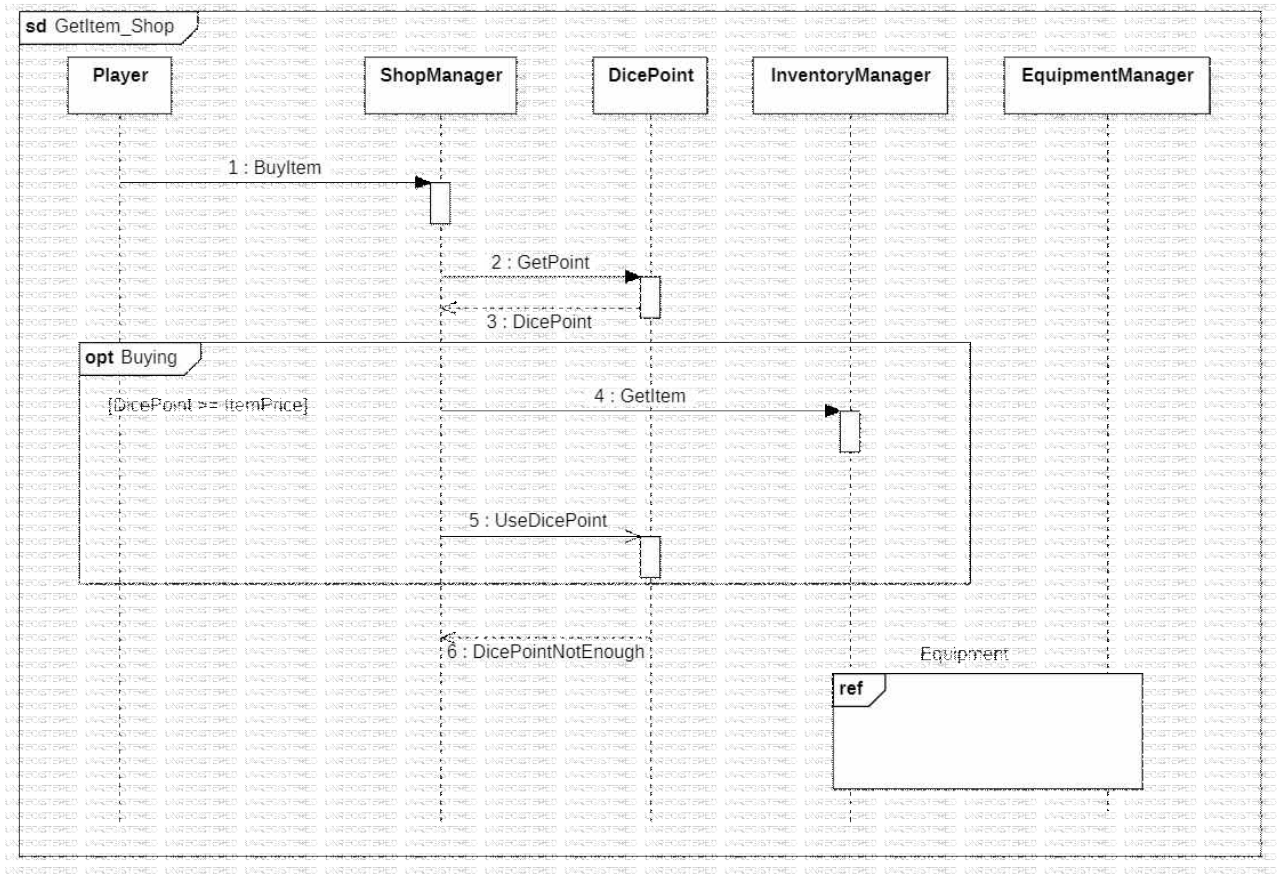
[그림 4-5] - GetItem_Drop

위 [그림 4-5]는 사용자가 적 처치를 통해 아이템이 드롭되는 Use Case를 나타내는

Sequence diagram이다.

사용자가 아이템이 드롭된 칸으로 이동해 GetItem 신호를 보내면 InventoryManager에서는 사용자가 아이템을 해당 부위에 착용하고 있는지 아닌지를 파악하고 내부적으로 ItemDrop 신호를 통해 장비 교체 여부를 판단한다. 장비 교체 또는 장비 습득을 하게 될 경우 4.4.의 Equipment 과정을 통해 장비를 착용하게 된다.

4.6. GetItem_Shop



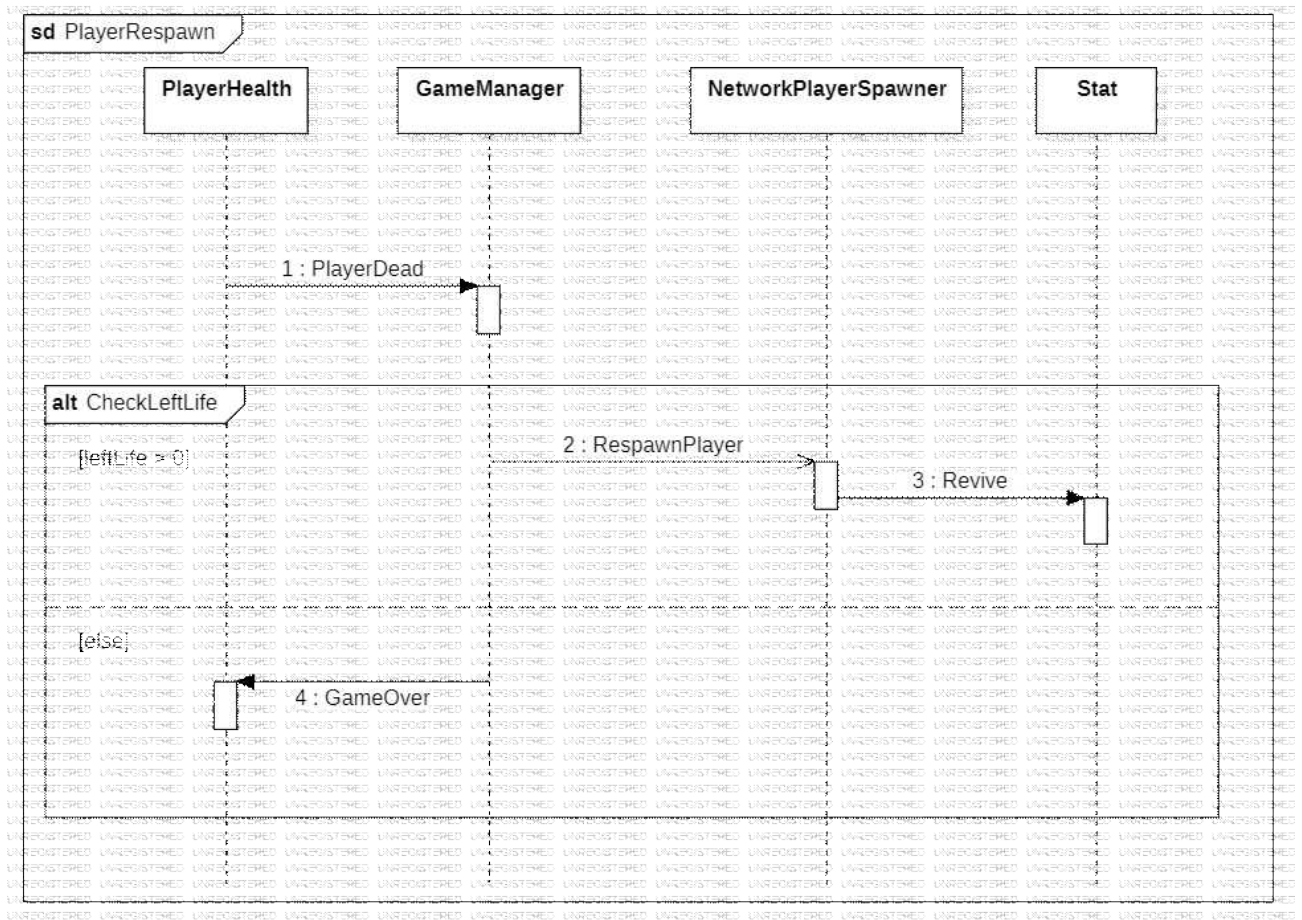
[그림 4-6] - GetItem_Shop

위 [그림 4-6]은 사용자가 상점을 통해 아이템이 획득하는 Use Case를 나타내는 Sequence diagram이다.

사용자가 상점을 통해 구매할 아이템을 선택하면 BuyItem 신호를 통해 ShopManager 클래스에서는 GetPoint 함수를 통해 DicePoint 클래스에서 플레이어가 보유한 포인트를 가져온다. 보유 포인트가 아이템 요구 포인트보다 많다면 아이템 구매가 성사되어 GetItem 신호를 통해 InventoryManager에게 아이템을 전달하고, UseDicePoint 함수를 통해 포인트를 소모한다. 만약 보유 포인트가 적은 경우에는 DicePointNotEnough 신호를 전달하여 아이템 구매가 불가능함을 알린다. GetItem 신호를 받은 InventoryManager

는 3.4.의 Equipment를 실행하여 아이템을 착용한다.

4.7. PlayerRespawn

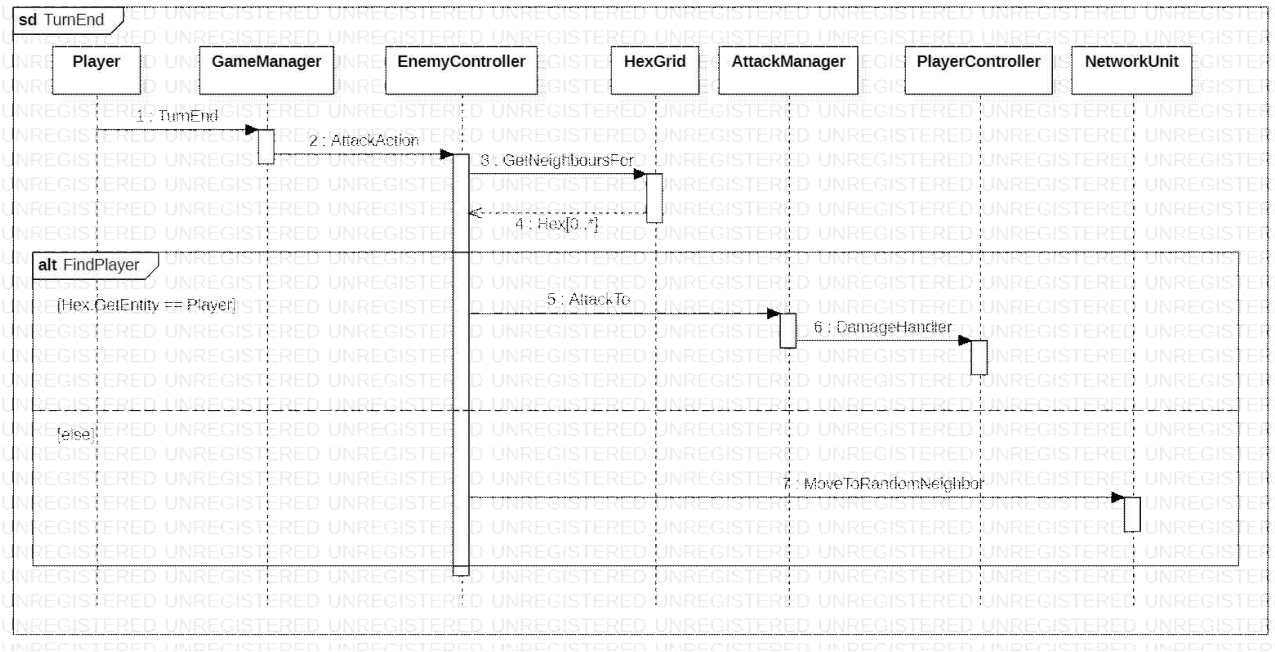


[그림 4-7] - PlayerRespawn

위 [그림 4-7]은 Player가 죽은 경우 부활하는 Use case를 나타내는 Sequence diagram이다.

PlayerHealth가 0이 되면 PlayerDead 신호를 GameManager에게 보낸다. 여기서 PlayerHealth는 플레이어의 체력을 나타내는 개념이다. 플레이어 사망 신호를 받은 GameManager는 남은 생명 수를 체크하여 생명이 남아있다면 NetworkPlayerSpawner에게 신호를 보내 NetworkPlayerSpawner가 Stat 클래스의 Revive 함수를 통해 Player를 부활시키고, 남은 생명이 없다면 GameOver 메시지를 Player에게 보여준다.

4.8. TurnEnd

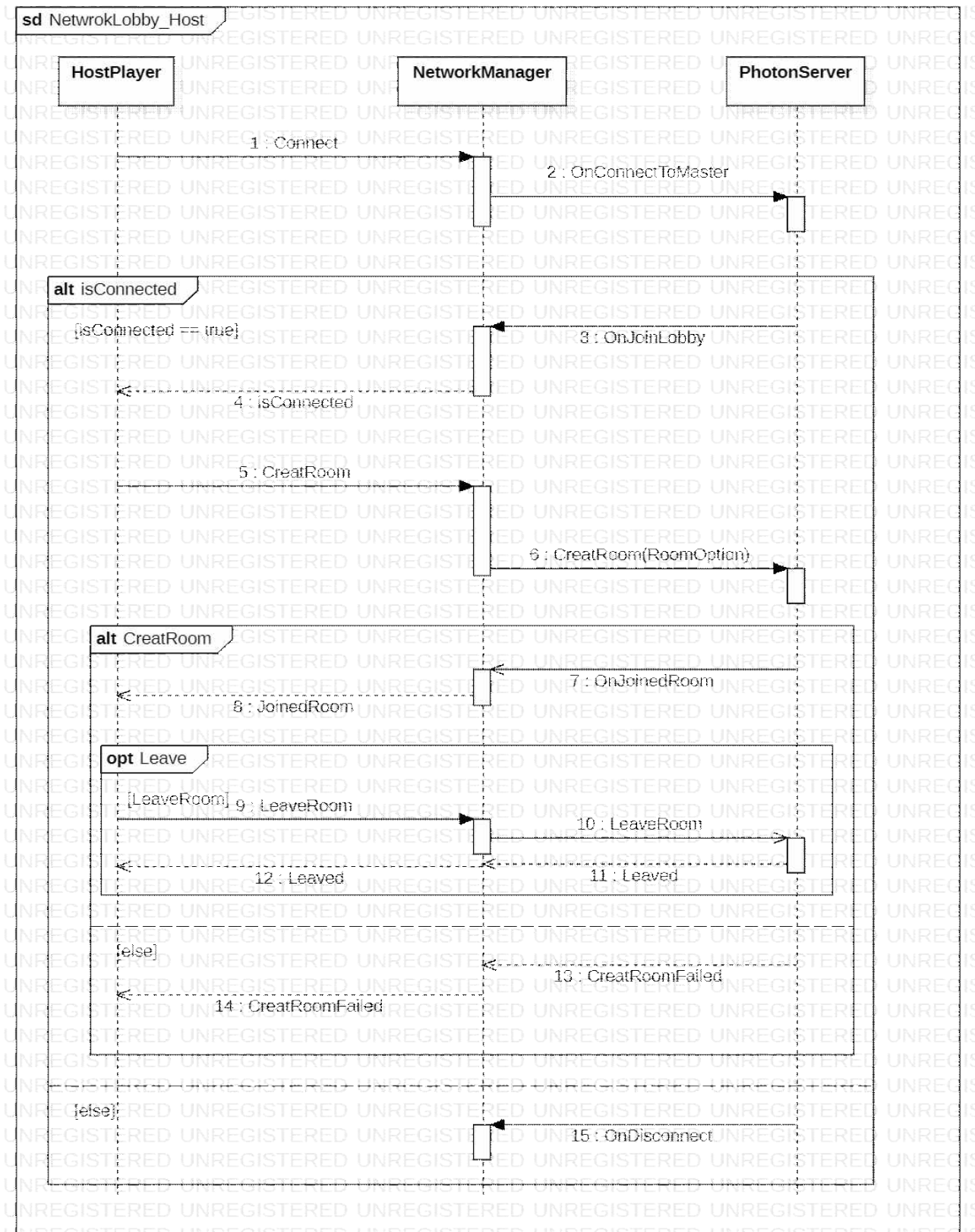


[그림 4-8] - TurnEnd

위 [그림 4-8]은 사용자가 자신의 턴을 마치는 Use case를 나타내는 Sequence diagram이다.

플레이어가 턴을 마치는 TurnEnd 버튼을 누르면 TurnEnd 함수를 통해 GameManager에서는 플레이어 턴을 종료하고 EnemyController에 AttackAction 함수를 통해 적 행동할 수 있도록 한다. EnemyController에서는 GetNeighboursFor 함수를 통해 플레이어의 위치가 적의 공격 범위에 있는지를 판단한다. 플레이어가 적의 공격 범위에 있다면 EnemyController 클래스는 AttackManager의 AttackTo 함수를 통해 플레이어에게 공격을 실행하고, AttackManager 클래스는 DamageHandler를 통해 PlayerController에 데미지 산정을 한다. 만약 플레이어가 적 공격 범위에 없다면 적은 MoveToRandomNeighbor 함수를 통해 주변 타일 중 한 군데로 랜덤하게 이동한다.

4.9. NetworkLobby_Host



[그림 4-9] - NetworkLobby_Host

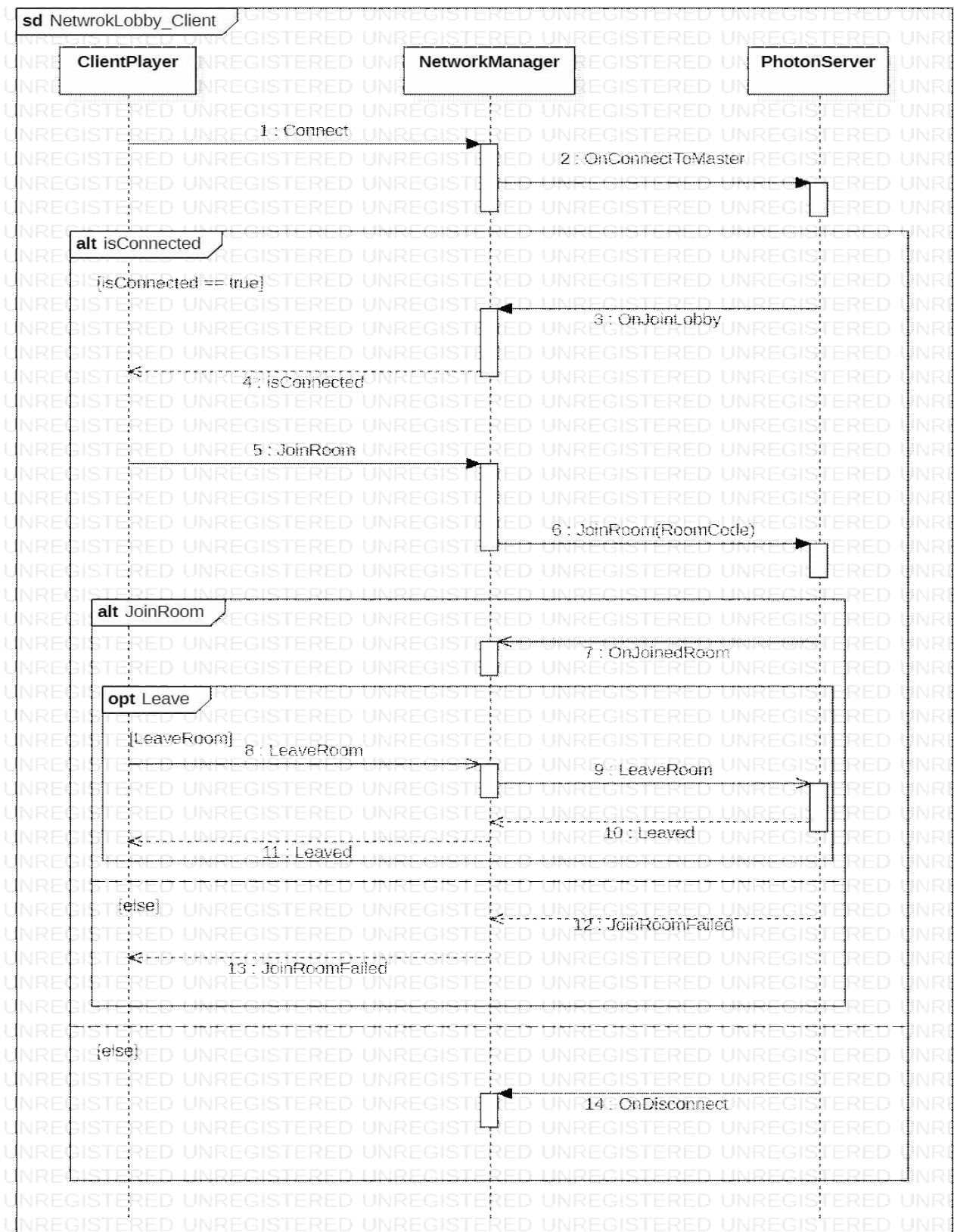
[그림 4-9]는 네트워크 플레이 Use case에서 방을 만드는 기능인 Host 부분을 나타낸 Sequence diagram이다.

Host Player가 NetworkManager에 Connect 요청을 하면 NetworkManager는 Photon Server에 OnConnectToMaster 신호를 통해 서버 접속 요청을 한다. 서버 접속이 정상적으로 이루어지면 Photon Server가 OnJoinLobby 메시지를 NetworkManager에 보내고 NetworkManager은 Host Player에게 연결이 완료되었다는 메시지를 보낸다. 만약 연결이 이루어지지 않았다면 Photon Server는 OnDisconnect 메시지를 통해 연결이 되지 않았다는 메시지를 보낸다.

연결이 정상적으로 완료되었으면 CreateRoom 신호를 Host Player가 NetworkManager에게 전달하고, NetworkManager는 해당 신호를 통해 Photon Server에서 방을 만들게 된다. Photon Server가 방을 만들면 OnJoinedRoom 신호를 보내고 그 신호를 받은 NetworkManager가 플레이어에게 신호를 주면 플레이어는 생성한 방에 들어오게 된다. 만일 방 생성에 실패했다면 Photon Server가 NetworkManager를 통해 플레이어에게 CreateRoomFailed 신호를 전달한다.

방을 떠나기 위해 플레이어가 LeaveRoom 신호를 NetworkManager를 통해 Photon Server에 전달하면 Photon Server는 방을 떠난 결과를 Leaved 메시지를 NetworkManager를 통해 플레이어에게 전달한다.

4.10 NetworkLobby_Client



[그림 4-10] - NetworkLobby_Client

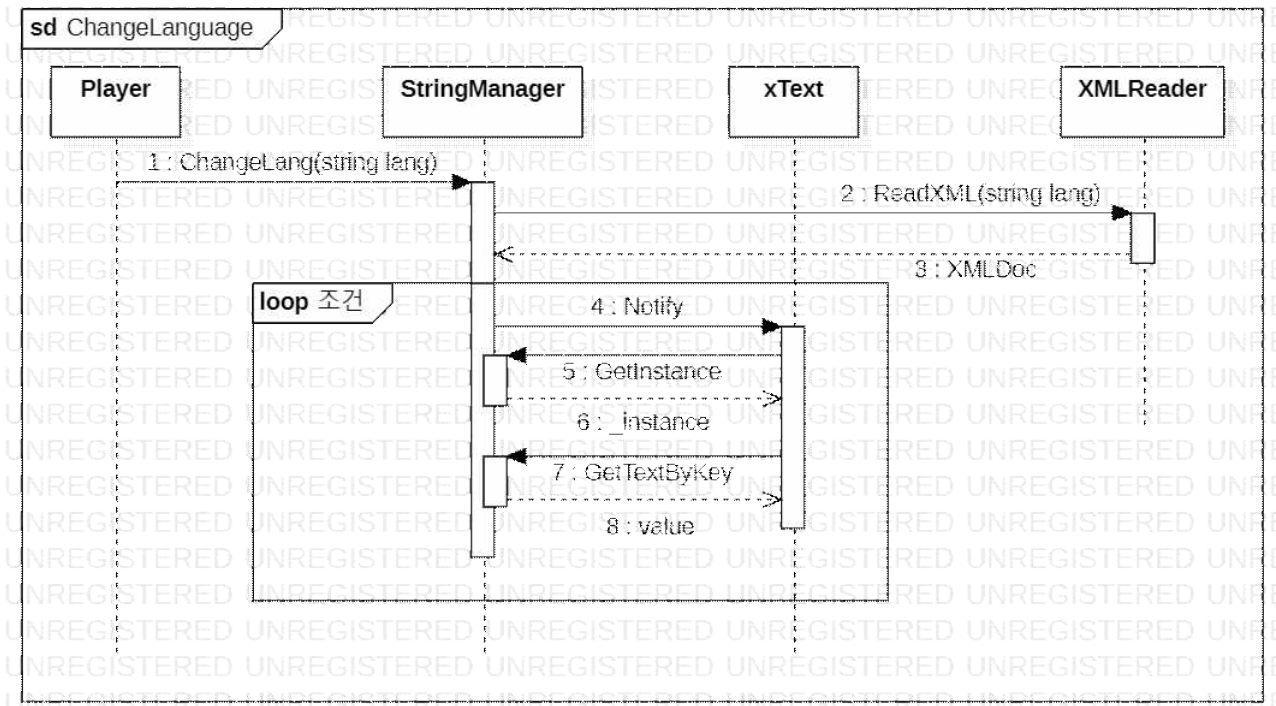
[그림 4-10]은 네트워크 플레이 Use case에서 방에 참가하는 기능인 Client 부분을 나타낸 Sequence diagram이다.

Client Player가 NetworkManager에 Connect 요청을 하면 NetworkManager는 Photon Server에 OnConnectToMaster 신호를 통해 서버 접속 요청을 한다. 서버 접속이 정상적으로 이루어지면 Photon Server가 OnJoinLobby 메시지를 NetworkManager에 보내고 NetworkManager는 Client Player에게 연결이 완료되었다는 메시지를 보낸다. 만약 연결이 이루어지지 않았다면 Photon Server는 OnDisconnect 메시지를 통해 연결이 되지 않았다는 메시지를 보낸다.

연결이 정상적으로 완료되었으면 JoinRoom 신호를 Client Player가 NetworkManager에게 전달하고, NetworkManager는 해당 신호와 함께 RoomCode를 통해 Photon Server에서 선택한 방을 찾고 방에 들어가게 된다. 방에 들어오면 Photon Server가 NetworkManager에 OnJoinedRoom 신호를 보내고, 방에 들어오기 실패했다면 Photon Server가 NetworkManager를 통해 플레이어에게 JoinRoomFailed 신호를 전달한다.

방을 떠나기 위해 플레이어가 LeaveRoom 신호를 NetworkManager를 통해 Photon Server에 전달하면 Photon Server는 방을 떠난 결과를 Leaved 메시지를 NetworkManager를 통해 플레이어에게 전달한다.

4.11. ChangeLanguage

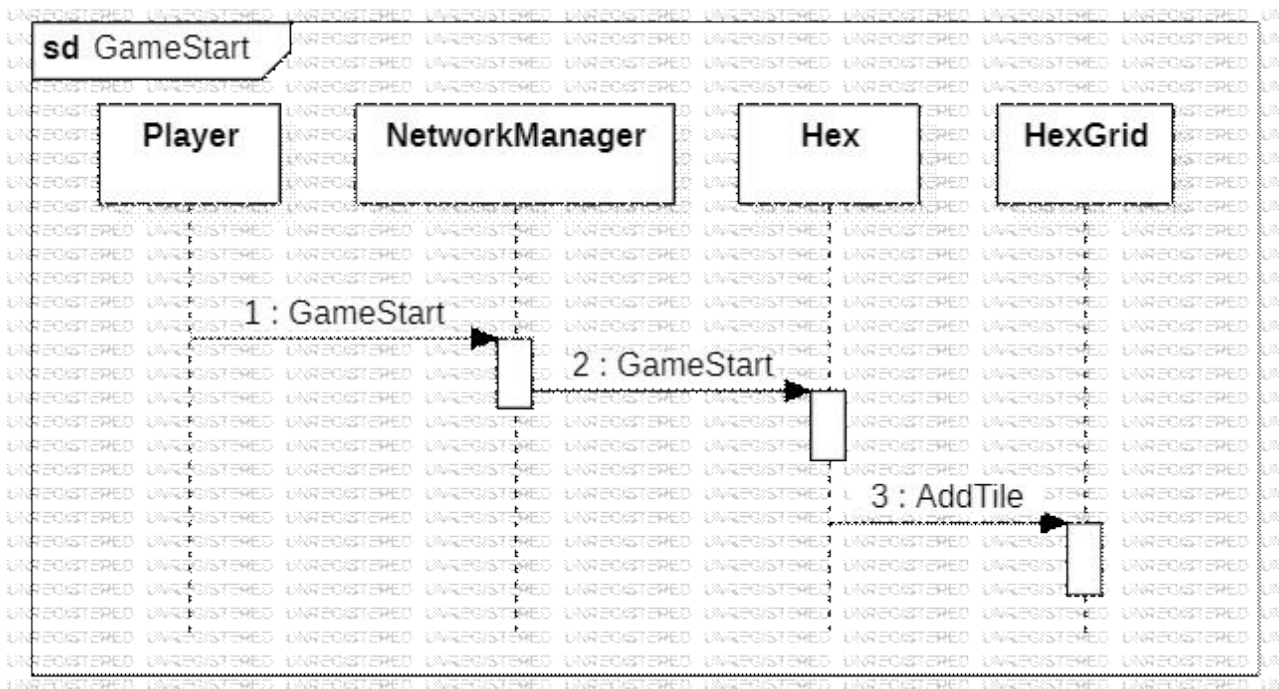


[그림 4-11] - ChangeLanguage

위 [그림 4-11]은 언어 변경 Use case를 나타내는 Sequence diagram이다.

플레이어가 언어 변경으로 원하는 언어를 선택하면 해당 언어가 StringManager에 전달되고, StringManager는 XMLReader를 통해 언어 XML 파일을 가져온다. Stringmanager는 xText에 Notify 함수를 이용하여 선택한 언어로 변경하게 된다. xText에서는 언어 변경을 위해 GetInstance 함수를 통해 StringManager에서 인스턴스를 가져오고, GetTextByKey 함수를 통해 각 언어에 대한 value 값을 가져와 언어 변경을 한다.

4.12. GameStart

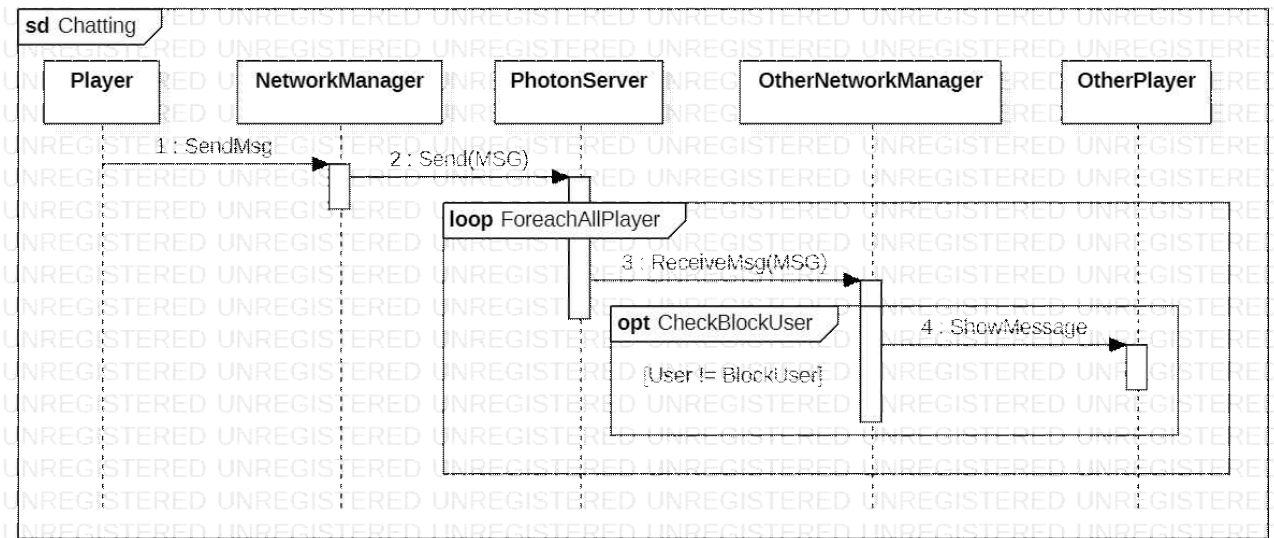


[그림 4-12] - GameStart

위 [그림 4-12]는 플레이어가 게임 시작 Use case를 나타내는 Sequence diagram이다.

플레이어가 GameStart 신호를 보내면 NetworkManager는 GameStart 신호를 Hex 클래스에 전달한다. Hex 클래스가 GameStart 신호를 수신하면 AddTile 함수를 통해 HexGrid 클래스에서 맵을 자동으로 불러오고 게임이 시작된다.

4.13. Chatting

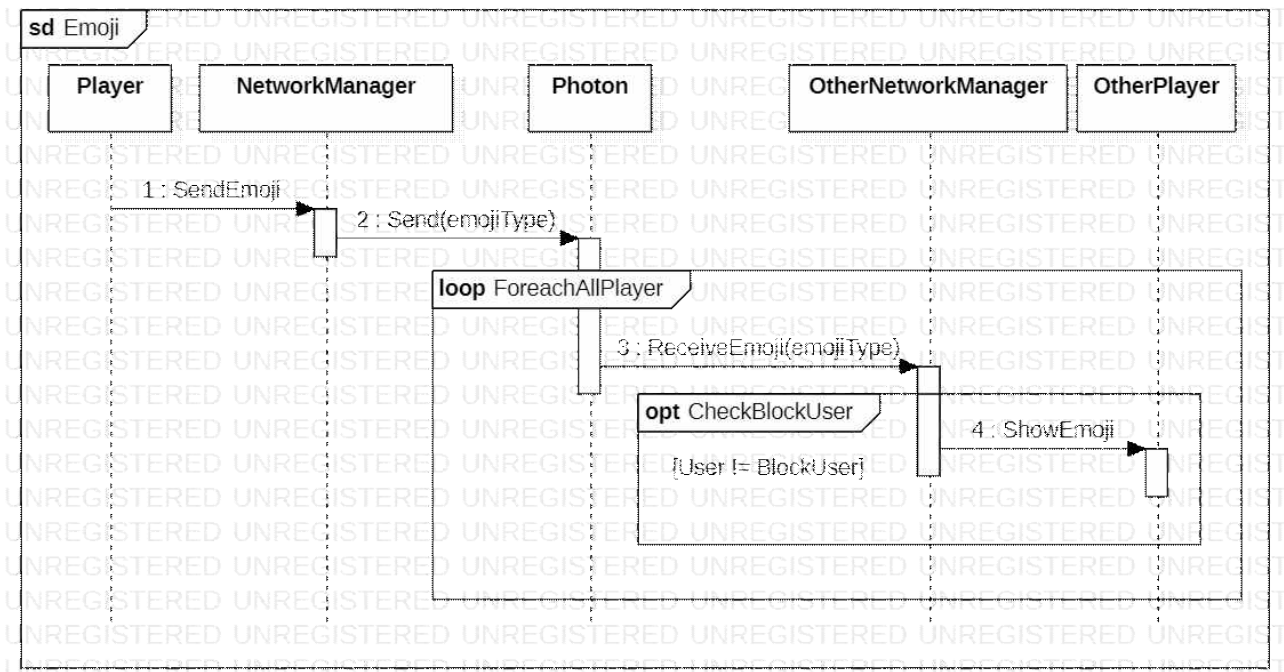


[그림 4-13] - Chatting

위 [그림 4-13]은 플레이어 채팅 Use case를 나타내는 Sequence diagram이다.

플레이어가 채팅 내용을 입력하고 채팅을 보내는 SendMsg 함수를 통해 채팅을 보내면 NetworkManager에서 해당 내용을 Send 함수를 통해 PhotonServer로 보낸다. PhotonServer에서는 해당 채팅 내용을 ReceiveMsg 함수를 통해 다른 플레이어의 NetworkManager로 보내고 채팅을 수신한 NetworkManager에서는 보낸 플레이어가 차단된 플레이어인지를 확인한 후 차단된 플레이어가 아니라면 수신한 채팅을 다른 플레이어에게 보여주게 된다.

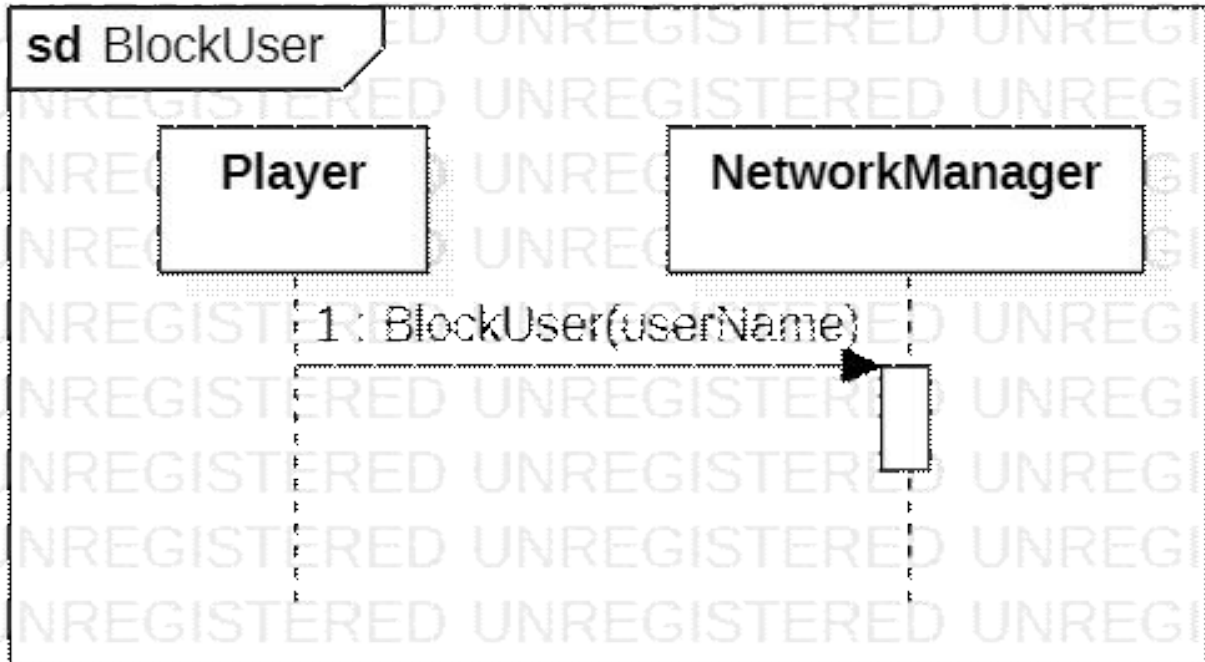
4.14. Emoji



[그림 4-14] - Emoji

위 [그림 4-14]은 플레이어 상호작용 Use case를 나타내는 Sequence diagram이다. 플레이어가 이모티콘을 선택하고 보내는 SendEmoji 함수를 통해 이모티콘을 보내면 NetworkManager에서 해당 이모티콘을 Send 함수를 통해 PhotonServer로 보낸다. PhotonServer에서는 해당 이모티콘을 ReceiveEmoji 함수를 통해 다른 플레이어의 NetworkManager로 보내고 이모티콘을 수신한 NetworkManager에서는 보낸 플레이어가 차단된 플레이어인지를 확인한 후 차단된 플레이어가 아니라면 수신한 이모티콘을 다른 플레이어에게 보여지게 된다.

4.15. BlockUser

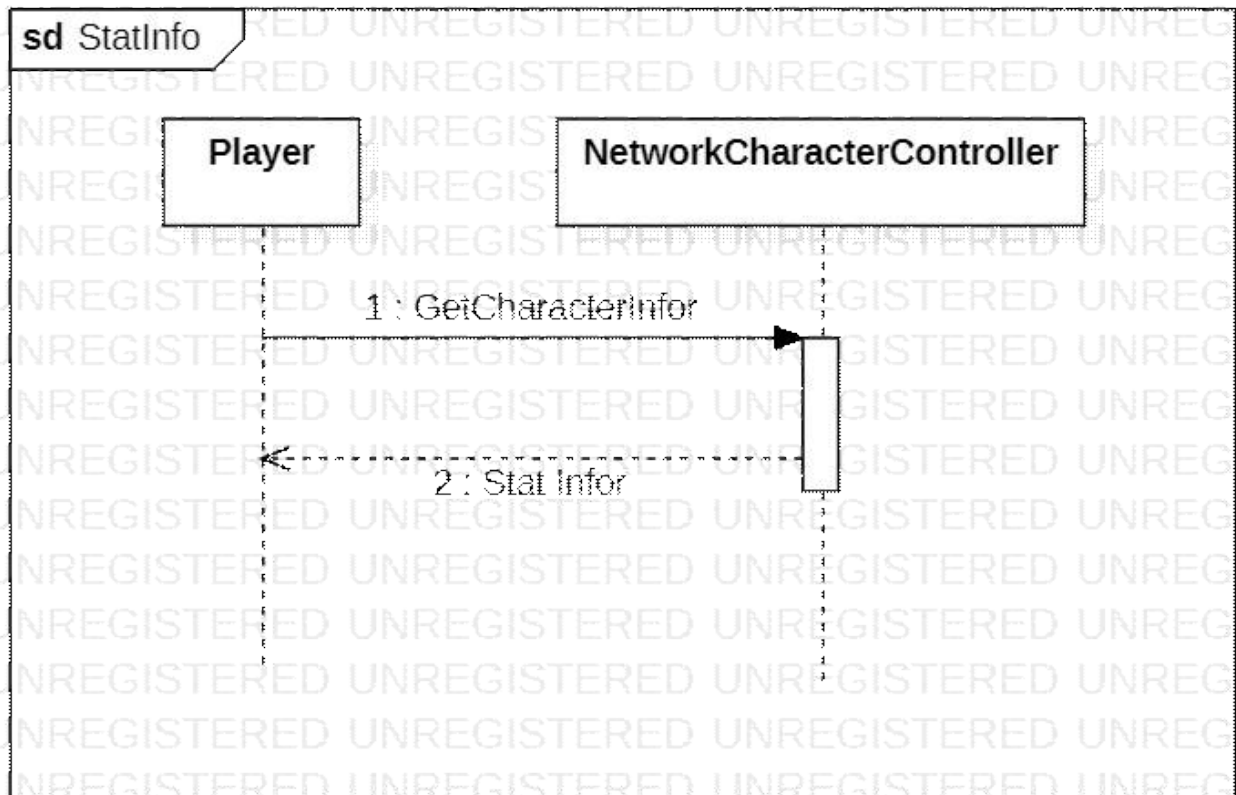


[그림 4-15] - BlockUser

위 [그림 4-15]는 다른 플레이어의 채팅 및 상호작용을 차단하는 Use case를 나타내는 Sequence diagram이다.

Player가 차단 명령어를 통해 차단 플레이어의 이름을 입력하면 NetworkManager의 BlockUser 함수를 통해 입력한 플레이어의 채팅 및 상호작용이 차단된다.

4.16. StatInfo

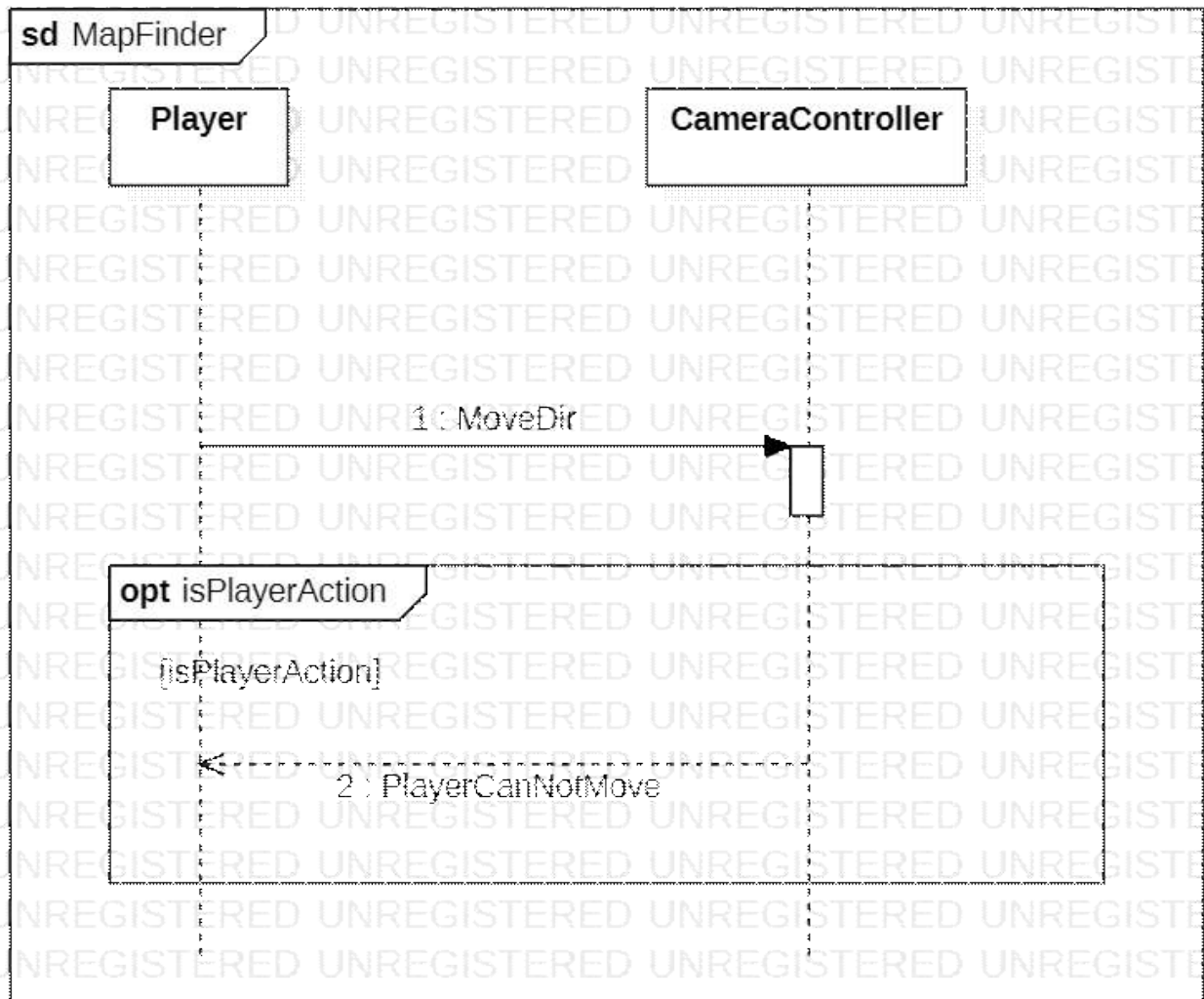


[그림 4-16] - StatInfo

위 [그림 4-16]는 캐릭터 스탯 확인을 위한 Use case를 나타내는 Sequence diagram 이다.

플레이어가 플레이어 및 적 캐릭터를 선택하면 선택한 캐릭터에 대해 GetCharacter Infor 함수를 NetworkCharacterController 클래스가 받아서 선택한 캐릭터에 대한 스탯을 플레이어에게 보여준다.

4.17. MapFinder

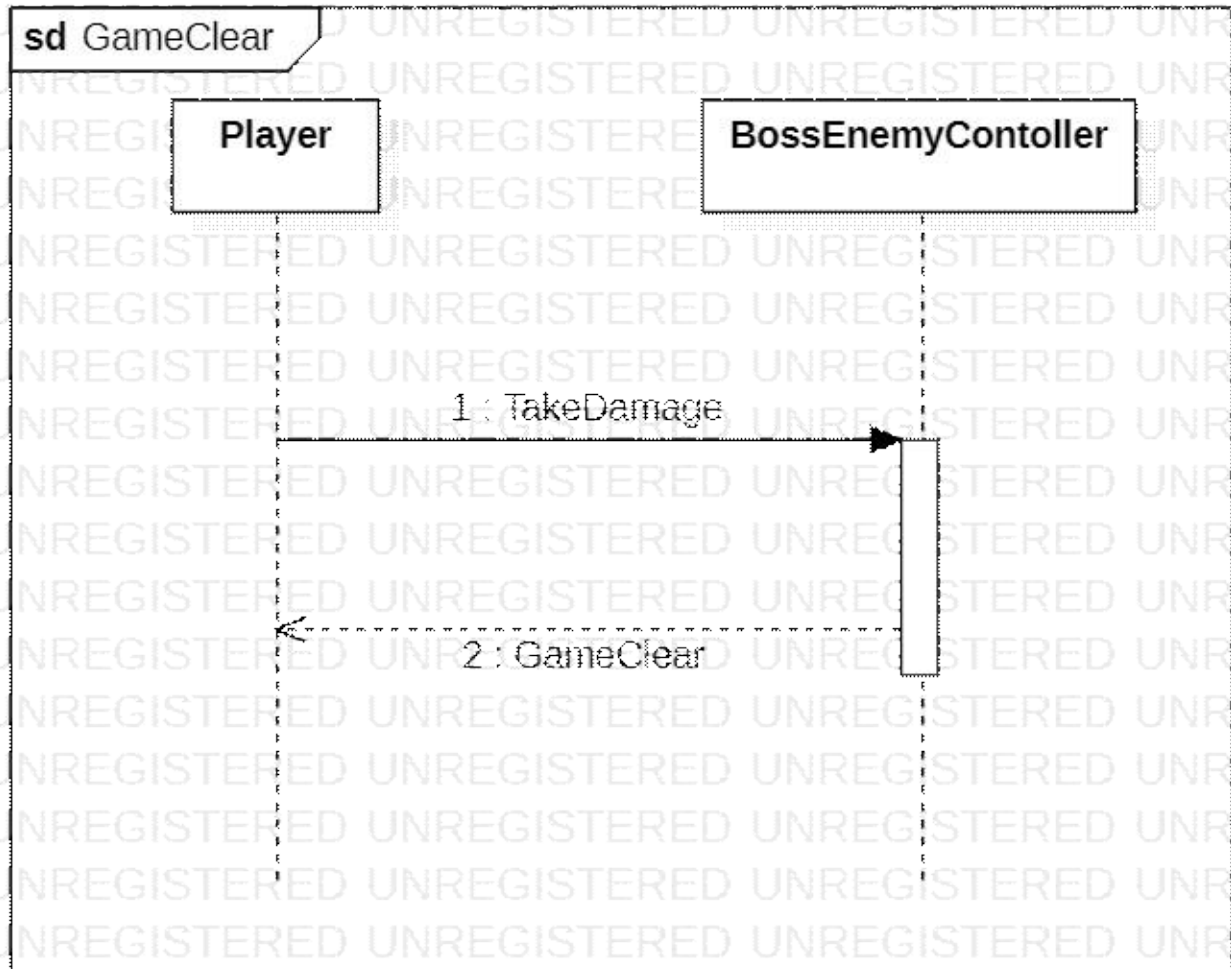


[그림 4-17] - MapFinder

위 [그림 4-17]은 맵 탐색 Use case를 나타내는 Sequence diagram이다.

플레이어가 탐색하고 싶은 방향을 인식하면 MoveDir 함수를 통해 CameraController 클래스가 카메라를 해당 방향으로 이동시켜 맵 탐색이 가능하도록 만든다. 맵 탐색 중에는 플레이어 행동이 불가능하기 때문에 맵 탐색 중에는 PlayerCanNotMove 신호를 통해 플레이어의 행동을 막는다.

4.18. GameClear



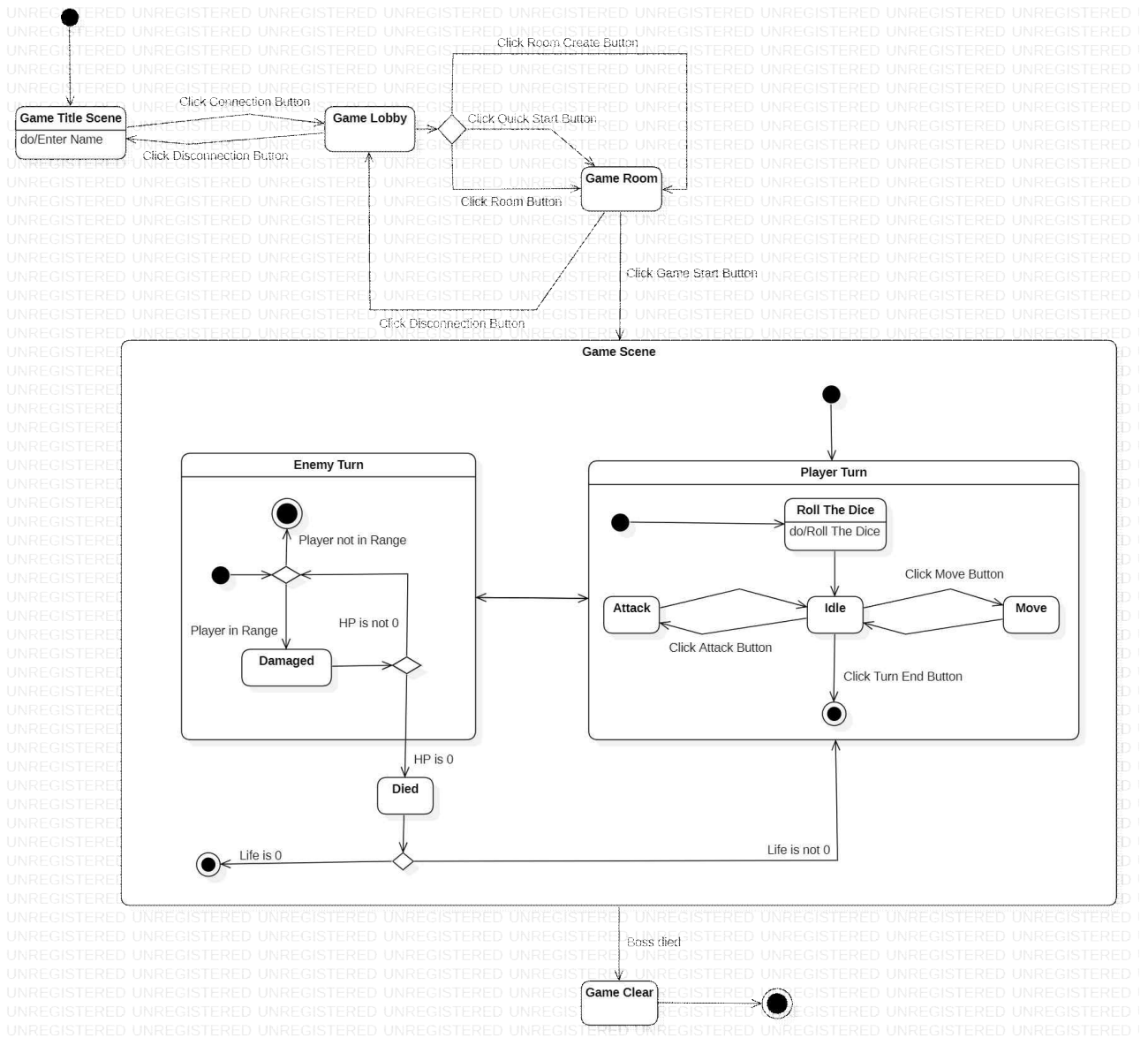
[그림 4-18] - GameClear

위 [그림 4-18]은 게임 클리어 Use case를 나타내는 Sequence diagram이다.

플레이어가 보스 몬스터를 쓰러뜨리면 TakeDamage 함수를 통해 BossEnemyController는 보스 몬스터의 죽음을 인식하고 게임 클리어 상태가 되었음을 플레이어에게 알려준다.

5. State machine diagram

이 장은 시스템의 State machine diagram을 그리고 설명한다. 아래 [그림 5-1]은 본 프로젝트에서 제작한 게임 시스템의 State machine diagram이다.



[그림 5-1] - State machine diagram

각 State는 게임에서 어떤 Scene을 보여주고 있는지에 대한 상태이고, Game Scene 내에서는 플레이어의 캐릭터가 어떠한 상태에 있는지를 나타내는 상태이다.

본 게임의 State는 크게 Game Title Scene, Game Lobby, Game Room, Game Scene, Game Clear로 흘러가는데 맨 처음 게임을 시작하게 되면 Game Title Scene 상태에서 게임을 시작하게 된다. Game Title Scene에서는 이름을 입력한 뒤 Connection 버튼, 즉 로그인 버튼을 누르는 것으로 게임에 접속할 수 있고, 게임에 접속하게 되면 Game Lobby 상태가 되어 게임 로비 화면이 표시된다. Game Lobby에서는 방 만들기 버튼을 눌러 게임 방을 생성해서 들어가거나, 빠른 시작 버튼을 눌러서도 게임 방을 생성해서 들어갈 수 있다. 또한 Room List에서 다른 사람이 만든 게임 방이 있다면 해당 게임 방 버튼을 누르는 것으로 다른 사람이 만든 게임 방에 들어갈 수 있다. 플레이어가 게임 방에 들어온 Game Room 상태에서는 Disconnection 버튼, 즉 방 나가기 버튼을 누르는 것으로 게임 방에서 나와 게임 로비로 돌아와서 Game Lobby 상태로 돌아갈 수 있다.

게임 방에서 게임 시작 버튼을 누르면 Game Scene이 나와 본격적으로 게임을 즐길 수 있다. Game Scene 상태는 내부적으로 Player Turn과 Enemy Turn을 중심으로 Player Turn 이전의 주사위 굴리기를 위한 Roll The Dice 상태와 Player Turn에서 선택한 행동으로 유발되는 Attack 및 Move 상태, Enemy Turn에 Enemy의 행동으로 유발되는 Damaged 상태와 그로 인한 Die 상태 및 Game Over 상태가 있다. Game Scene이 시작되면 Roll The Dice 상태에서 주사위를 클릭한 후 Player Turn 상태로 넘어간다. Player Turn 상태에서는 공격 버튼을 누르면 Attack 상태가, 이동 버튼을 누르면 Move 상태가 된다. 턴 종료 버튼을 누르면 Enemy Turn이 되고 Enemy Turn에 플레이어가 Damage를 입으면 Damaged 상태가 되어 Player의 HP를 검사한다. HP가 0이 되지 않으면 Enemy Turn으로 돌아가고 HP가 0이 되면 플레이어는 Die 상태가 되어 남은 목숨이 없다면 Game Over 상태가 되고 남은 목숨이 있다면 부활하여 Player Turn에 부활하기 위해 Respawn 한다. Enemy의 행동이 끝나면 Enemy Turn이 종료 되고 Roll The Dice 상태가 되어 다시 주사위를 굴리고 Player Turn으로 돌아가는 반복 상태가 된다. 게임의 보스 몬스터를 처치하면 Game Scene이 종료되어 Game Clear 상태가 되어 게임이 끝나게 된다.

6. User interface prototype

이 장은 예상 UI와 UI 안의 각 구성요소를 설명한다. 실제 개발에 따라서 UI 디자인은 일부 달라질 수 있지만 내용 및 구성은 동일하다. 본 문서에서 제공되는 UI 화면은 원활한 설명을 위해 언어 설정을 한국어로 설정했을 때를 기준으로 한다.

아래 [그림 6-1]은 게임을 맨 처음 실행하면 등장하는 타이틀 화면이다.



[그림 6-1] - 타이틀 화면

이름 입력 란에 이름을 입력한 후 로그인 버튼을 누르면 게임 로비 화면으로 이동한다.

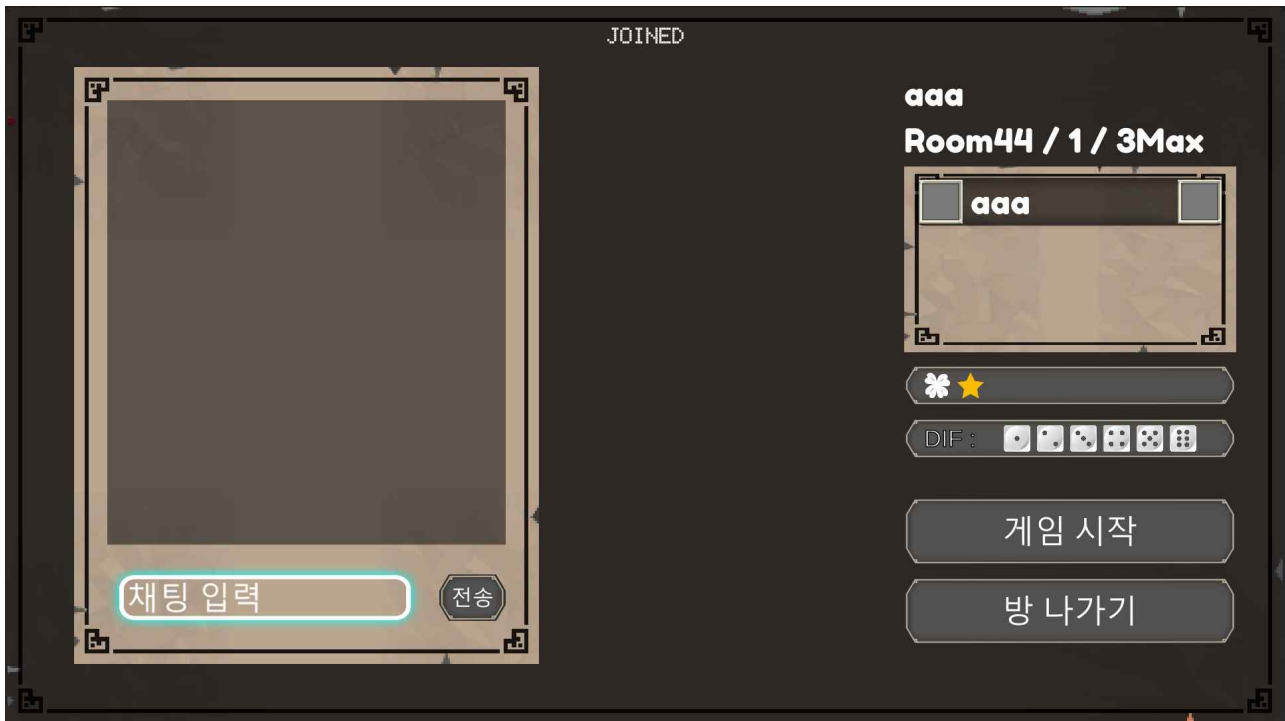
아래 [그림 6-2]는 게임 로비 화면이다. 아래 그림에서 타이틀 화면에서 aaa라는 이름을 입력한 후 로그인 버튼을 눌렀기 때문에 우측 리스트 상단을 보면 안녕하세요 aaa라는 문구가 표시된 것을 확인할 수 있다.



[그림 6-2] - 게임 로비 화면

위 로비 화면에서는 좌측 하단의 방 이름 란에 방 이름을 원하는 대로 입력한 후 방 만들기 버튼을 통해 방을 생성할 수도 있고, 빠른 시작 버튼을 통해 방 이름을 입력하지 않고 생성한 플레이어 본인의 이름으로 자동 생성되는 방을 만들 수도 있다. 또한, 우측 리스트에 표시되는 다른 사람이 만든 방을 하단의 좌우 화살표를 이용하여 탐색해 원하는 방을 클릭해서 들어갈 수도 있다. 위의 붉은 화살표 버튼을 누르면 게임 타이틀 화면으로 돌아간다.

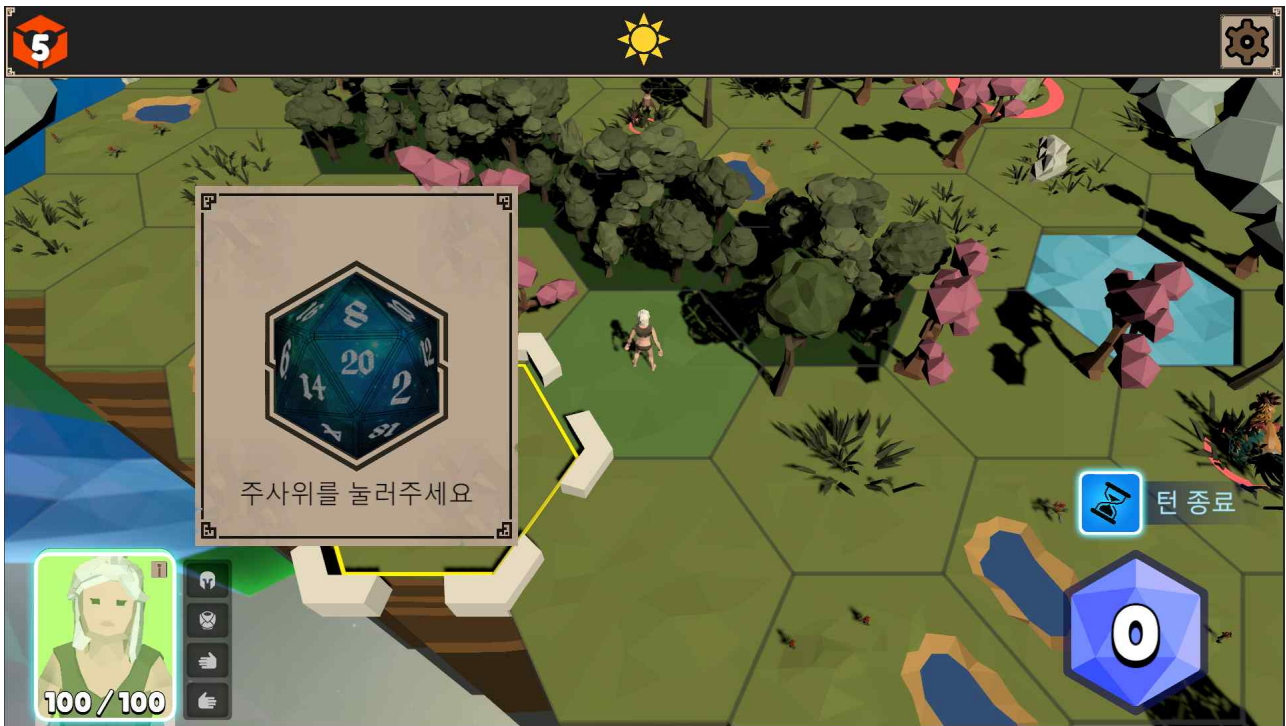
아래 [그림 6-3]은 게임 방에 들어오면 볼 수 있는 게임 방 UI이다.



[그림 6-3] - 게임 방 화면

좌측 채팅 입력란에 원하는 말을 입력하고 전송 버튼을 누르는 것으로 방에 있는 다른 플레이어와 채팅을 할 수 있으며, 우측에 있는 게임 시작 버튼을 누르는 것으로 게임을 시작할 수 있고, 방 나가기 버튼을 누르는 것으로 게임 방을 나와 로비로 돌아갈 수 있다. 게임 시작 버튼 위 DIF 리스트에서 주사위를 선택하면 선택한 주사위에 따라 기본 제공되는 주사위 포인트가 달라져 이 기본 포인트를 통한 난이도 조절이 가능하다. 만약 선택하지 않고 게임 시작 버튼을 누르게 되면 기본 제공되는 주사위 포인트가 없는 상태로 게임을 시작하게 된다. 우측 상단을 보면 플레이어 이름인 aaa 아래 방 이름인 Room44와 현재 방의 인원 1, 방의 최대 정원인 3이 Room44/1/3Max와 같은 형태로 표시되어 있는 것을 알 수 있다. 그 아래에는 방에 있는 인원이 리스트로 표시되고, 만약 방에 인원이 있다면 게임 시작 버튼이 준비하기 버튼으로 바뀌어 이름 우측 체크박스에 준비하기 버튼을 누르면 체크 표시가 표시되어 방에 있는 모든 인원이 준비가 되면 게임이 시작된다. 리스트 아래 바에는 이모티콘이 표시되는데, 해당 이모티콘을 클릭하면 선택한 이모티콘이 이름 리스트에서 자신의 이름 앞 박스에 이모티콘이 표시된다.

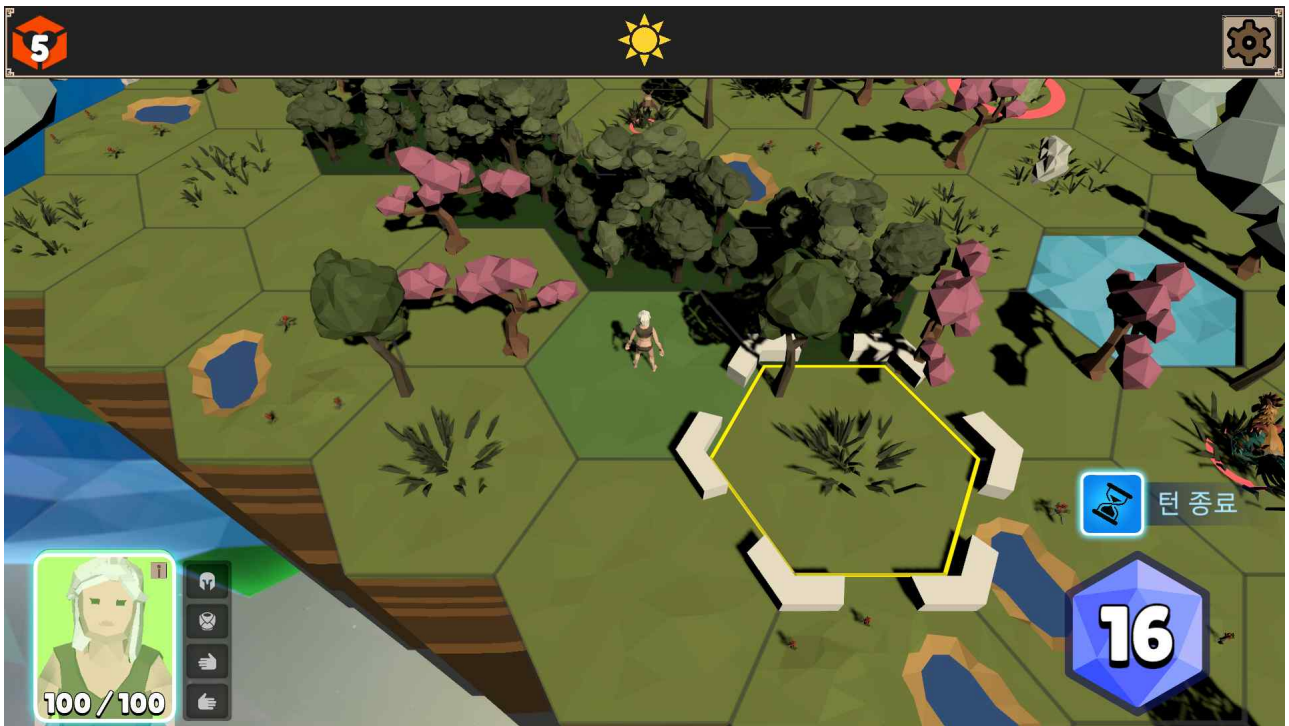
아래 [그림 6-4]는 게임 시작 시 플레이어 턴 시작 단계의 주사위를 굴리기 위한 UI이다.



[그림 6-4] - 주사위 굴리기

주사위 버튼을 클릭하면 주사위가 굴러가며 주사위의 눈이 우측 하단의 포인트로 가산이 된다.

아래 [그림 6-5]는 주사위를 굴려 나온 눈의 수가 포인트로 가산된 모습으로, 우측 하단의 숫자가 [그림 6-4]의 0에서 16으로 바뀐 것을 확인할 수 있다.



[그림 6-5] - 주사위 포인트가 가산된 모습

주사위를 굴러 포인트를 획득한 후 플레이어는 플레이어를 클릭하는 것으로 행동을 선택할 수 있는데, 아래 [그림 6-6]은 플레이어 행동 선택을 위해 플레이어를 선택한 모습이다.



[그림 6-6] - 플레이어 행동 선택

플레이어를 선택하면 공격, 이동, 탈기 세 가지 버튼이 나오고 탈기 버튼을 누르면 동작 선택 버튼 UI가 사라진다.

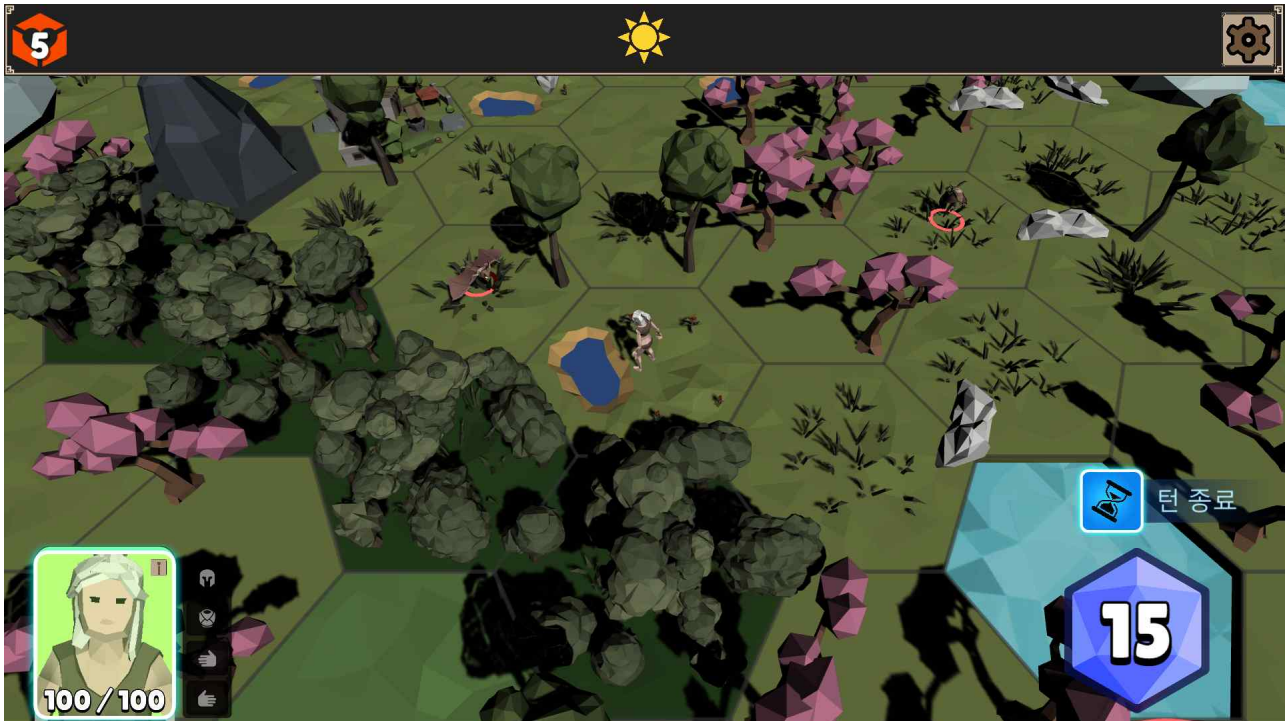
플레이어 공격을 위해 공격 버튼을 누르면 공격 가능한 필드 표시가 나타나는데, 아래 [그림 6-7]은 필드에 공격 가능 지역 UI가 표시된 모습이다.



[그림 6-7] - 공격 UI 표시

위 [그림 6-7]처럼 공격 가능 범위에 적이 있는 경우, 적이 있는 칸을 선택하는 것으로 공격할 적을 선택할 수 있고, Attack 버튼을 누르는 것으로 적에게 공격을 가할 수 있다. 적에게 공격을 가하면 가한 데미지만큼 적 아래 붉은 원으로 표시된 체력에 피해를 입힐 수 있다. 만약 공격을 취소하고 싶은 경우 Attack 버튼 옆의 Cancel 버튼을 누르면 공격이 취소된다.

아래 [그림 6-8]은 플레이어 공격 후 적의 체력이 소모된 것이다.



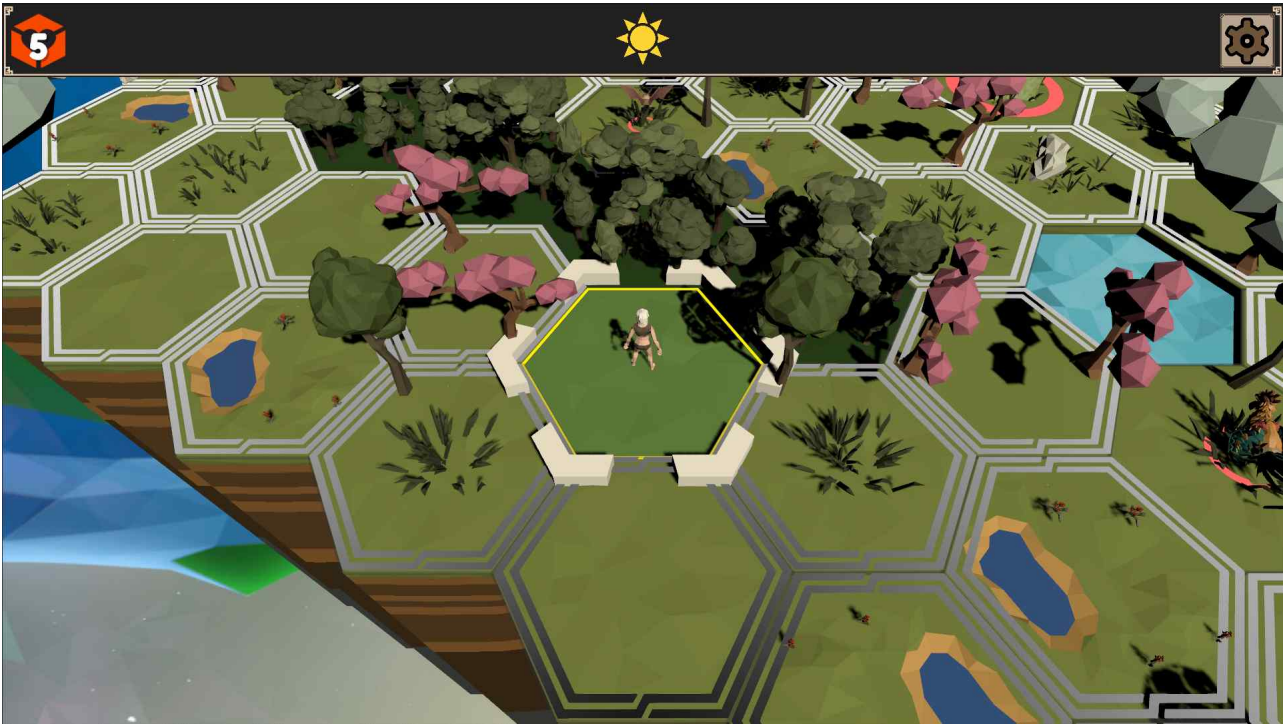
[그림 6-8] - 적 체력이 소모된 모습

또한 아래 [그림 6-9]를 보면 추가적으로 몬스터가 플레이어에게 공격을 입힌 경우, 좌측 하단의 캐릭터 아이콘 배경의 초록색으로 표시된 체력이 소모된 것을 같이 확인할 수 있다.



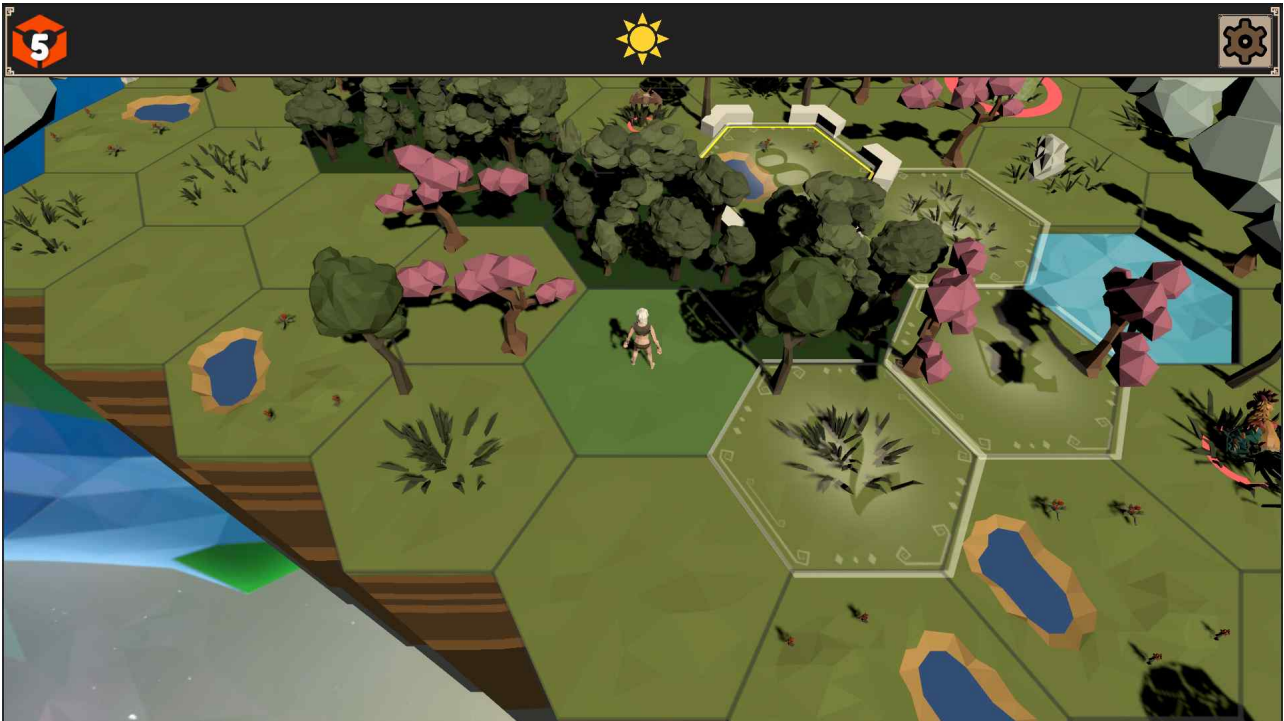
[그림 6-9] - 플레이어 체력이 소모된 모습

아래 [그림 6-10]은 플레이어 행동 선택에서 플레이어 이동을 위해 이동 버튼을 눌렀을 때 보유 포인트에 따라 플레이어가 이동이 가능한 타일이 표시된 UI이다.



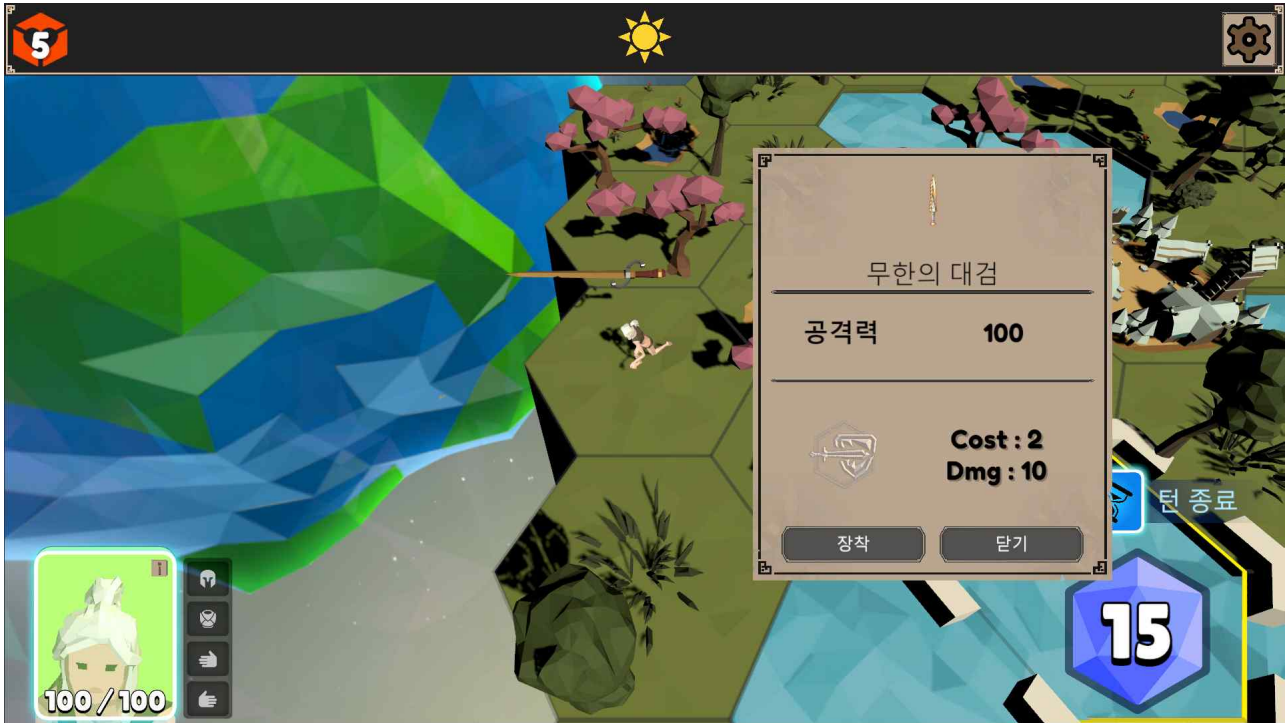
[그림 6-10] - 이동 가능한 타일 표시

이동이 가능한 타일 중 이동하고 싶은 타일을 선택하면 아래 [그림 6-11]과 같이 소모되는 포인트량이 타일에 표시된다. 이때 선택한 타일 외에 표시되는 타일의 경우 선택한 타일로 가는 이동 경로에 해당하는 타일이며, 각각 소모되는 포인트량이 누적되어 표시된다. 보유되는 소모 포인트를 확인하고 다시 해당 타일을 선택하면 포인트가 소모되고 캐릭터는 해당 타일로 이동한다.



[그림 6-11] - 이동 거리에 따른 소모 포인트 표시

적을 처치하였을 때 장비 아이템이 드롭된다면 해당 위치에 장비 아이템 오브젝트가 표시되고 플레이어가 해당 칸으로 이동하면 드롭된 장비를 획득할 수 있다. 아래 [그림 6-12]는 드롭된 장비를 획득할 때 장비의 정보를 표시하는 UI이다.



[그림 6-12] - 드롭된 장비 정보 확인

장착 버튼을 누르면 장비를 습득하여 착용하는데, 장비를 착용하게 되면 좌측 하단의 장비 아이콘에 장비 착용 표시가 나타나는 것을 확인할 수 있다. 탈기 버튼을 누르면 장비를 습득하지 않고 지나간다.



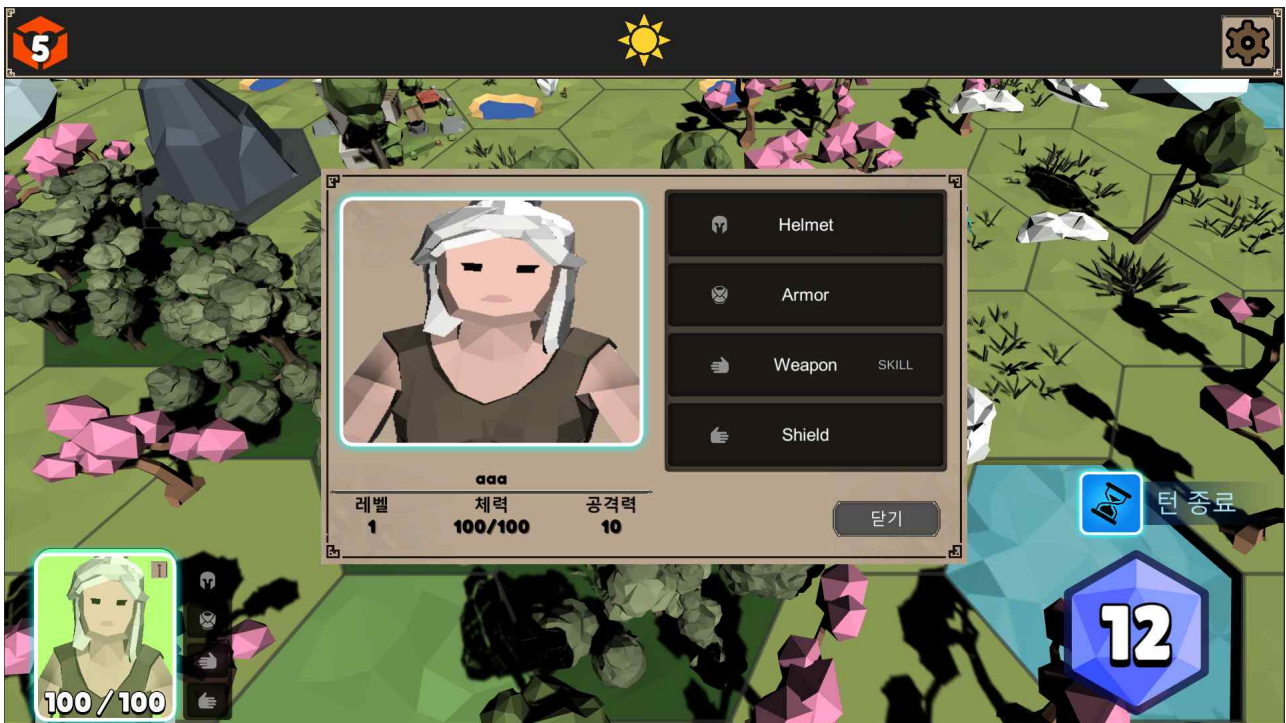
[그림 6-13(좌)] - 장비 착용 전

[그림 6-13(우)] - 장비 착용 후

위 [그림 6-13(좌)]는 장비를 착용하기 전, [그림 6-13(우)]는 장비 중 무기 부위를 착용

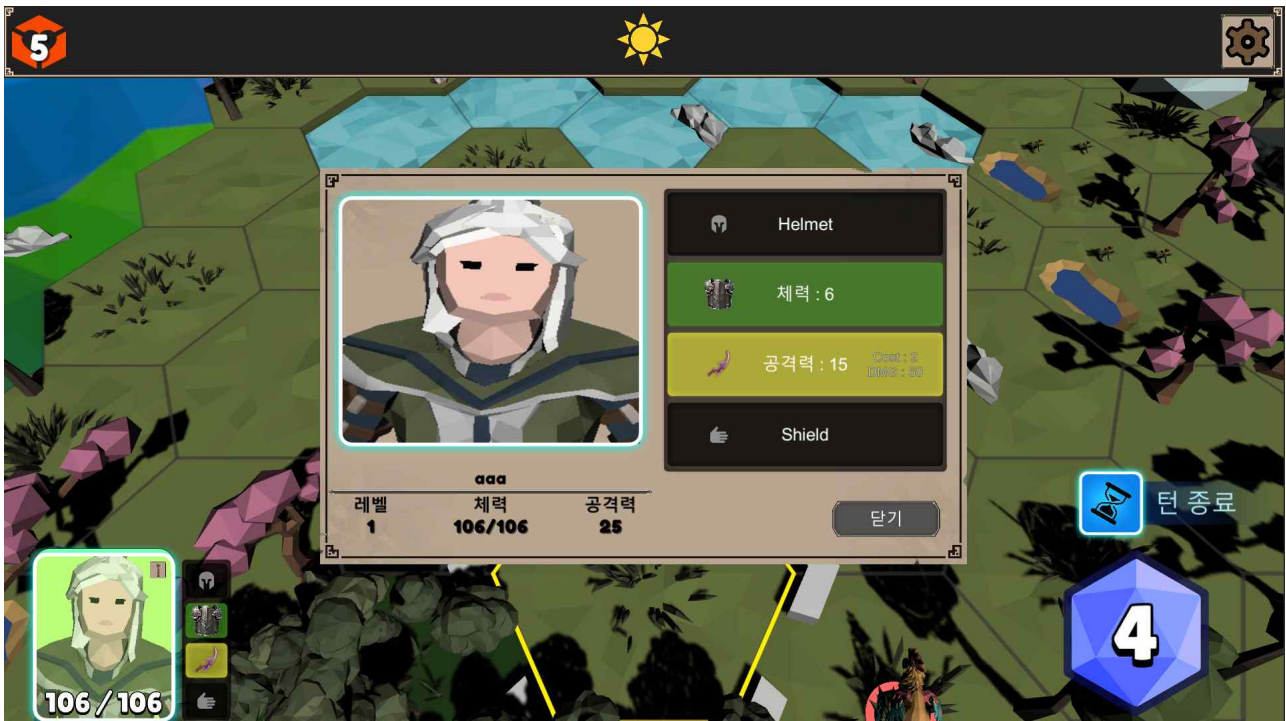
한 것으로 장비를 착용하게 되면 각 장비 부위에 해당하는 부분의 UI에 아이콘 형태로 표시되는 것을 확인할 수 있다.

또한 캐릭터 우측 상단에 있는 작은 버튼을 통해 플레이어의 현재 스펙을 확인할 수 있다. 아래 [그림 6-14]는 해당 버튼을 눌러 캐릭터 스펙을 확인하기 위한 UI를 표시한 것이다.



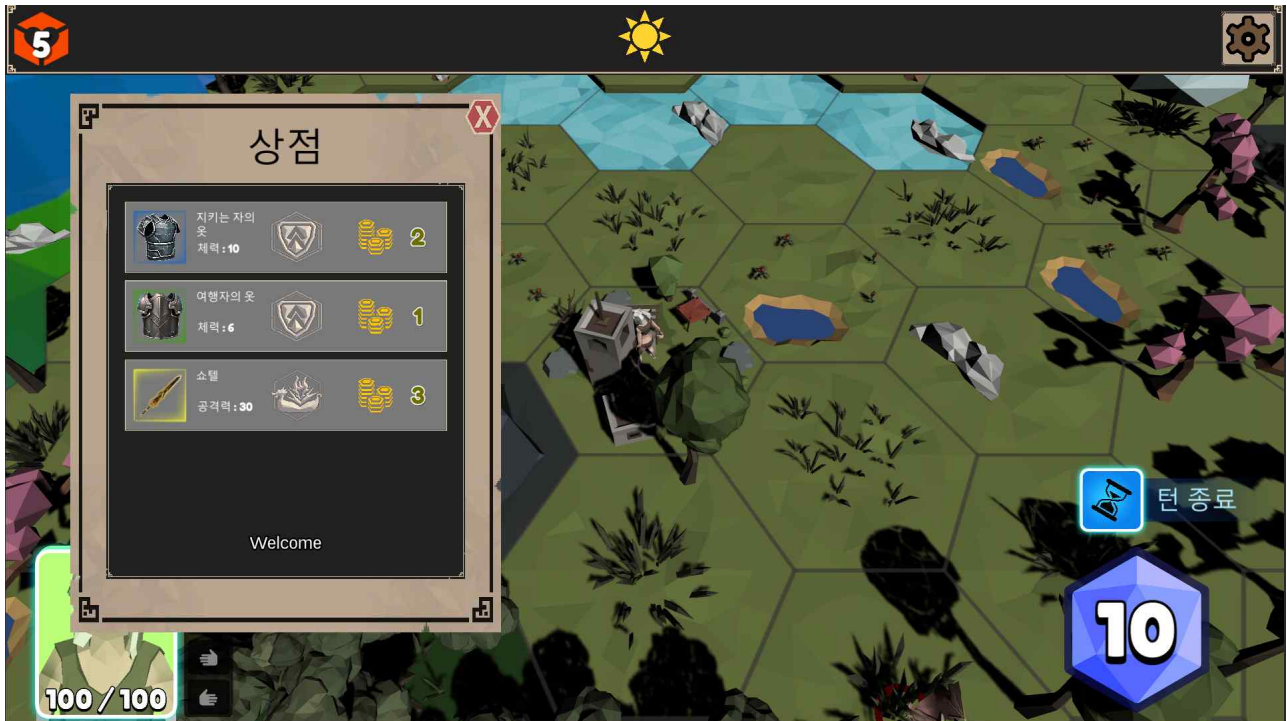
[그림 6-14] - 캐릭터 스펙 확인

만약 장비를 착용하고 있는 상태인 경우 아래 [그림 6-15]와 같이 확인창 우측의 장비 칸에서 착용하고 있는 장비의 정보를 확인할 수 있다.



[그림 6-15] - 장비 착용 캐릭터 스펙 확인

아래 [그림 6-16]은 마을 상점을 통해 장비를 구매할 수 있는 상점 UI이다. 해당 UI에는 장비의 정보와 구매에 필요한 포인트 비용이 표시되고, 이를 클릭하는 것으로 장비를 구매해 캐릭터가 착용하게 된다.



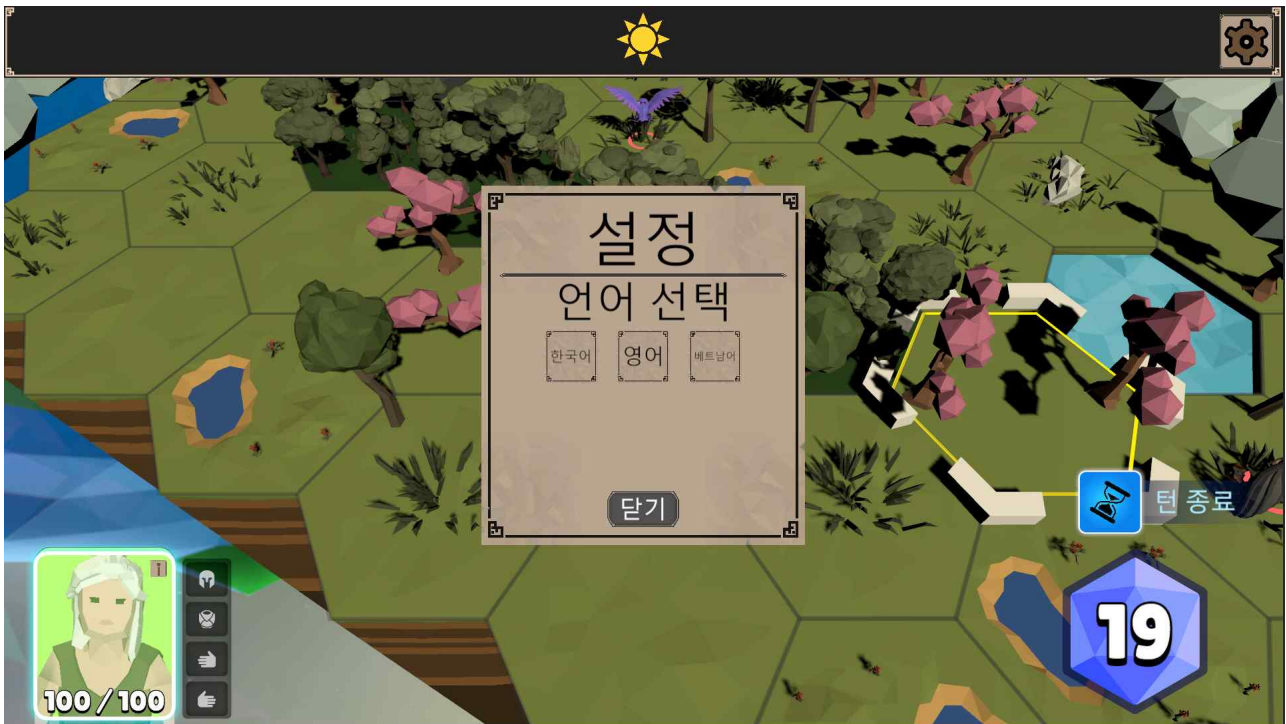
[그림 6-16] - 상점 UI

또한 몇몇 장비는 스킬 옵션을 보유하고 있어 착용 시 스킬을 사용할 수 있는데, 스킬을 획득하게 되면 플레이어 행동 선택에 스킬 버튼이 활성화되어 스킬 사용이 가능해진다. 아래 [그림 6-17]은 스킬 버튼이 활성화된 플레이어 행동 선택 UI이다.



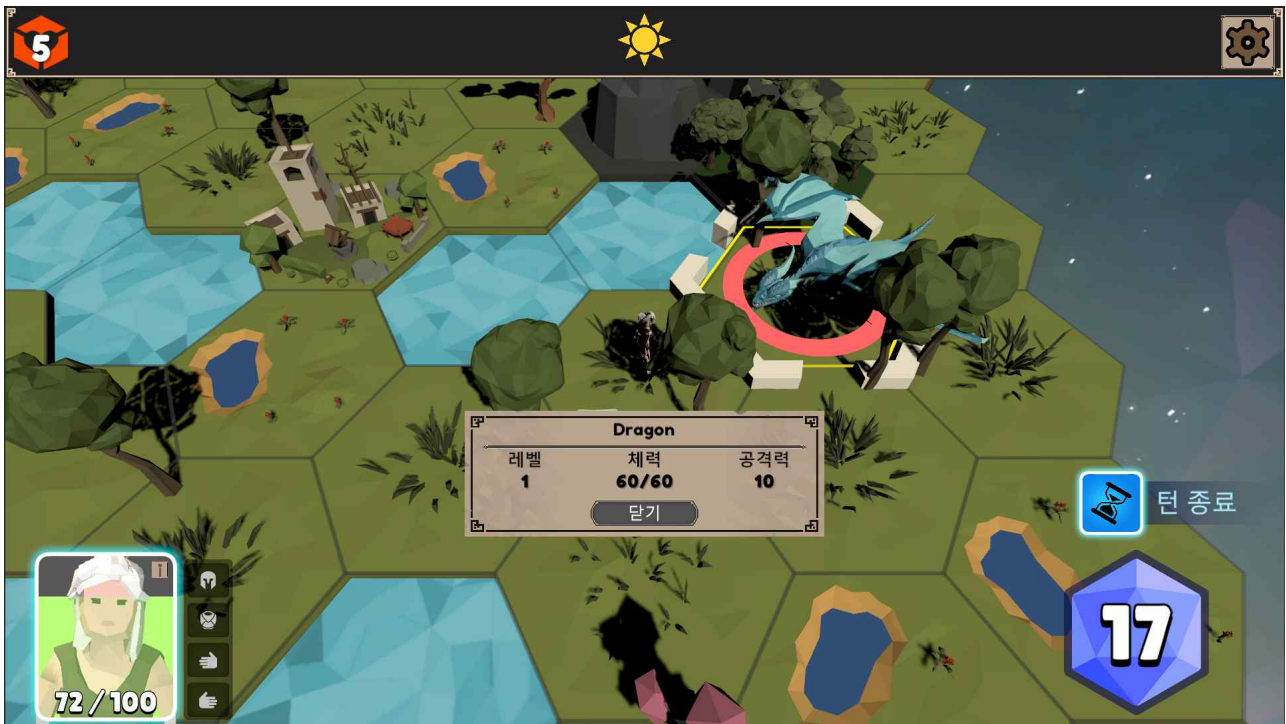
[그림 6-17] - 스킬 버튼 활성화

아래 [그림 6-18]는 언어 변경을 위한 설정 창 UI이다. 해당 UI의 언어 선택 아래에 있는 버튼을 통해 본 시스템에서 제공되는 UI 언어 변경이 가능하다. 변경 가능한 언어로는 영어, 한국어, 베트남어 변경이 가능하다. 해당 설정은 우측 상단의 태엽 모양 아이콘으로 나타난 설정 버튼을 누르면 표시된다.



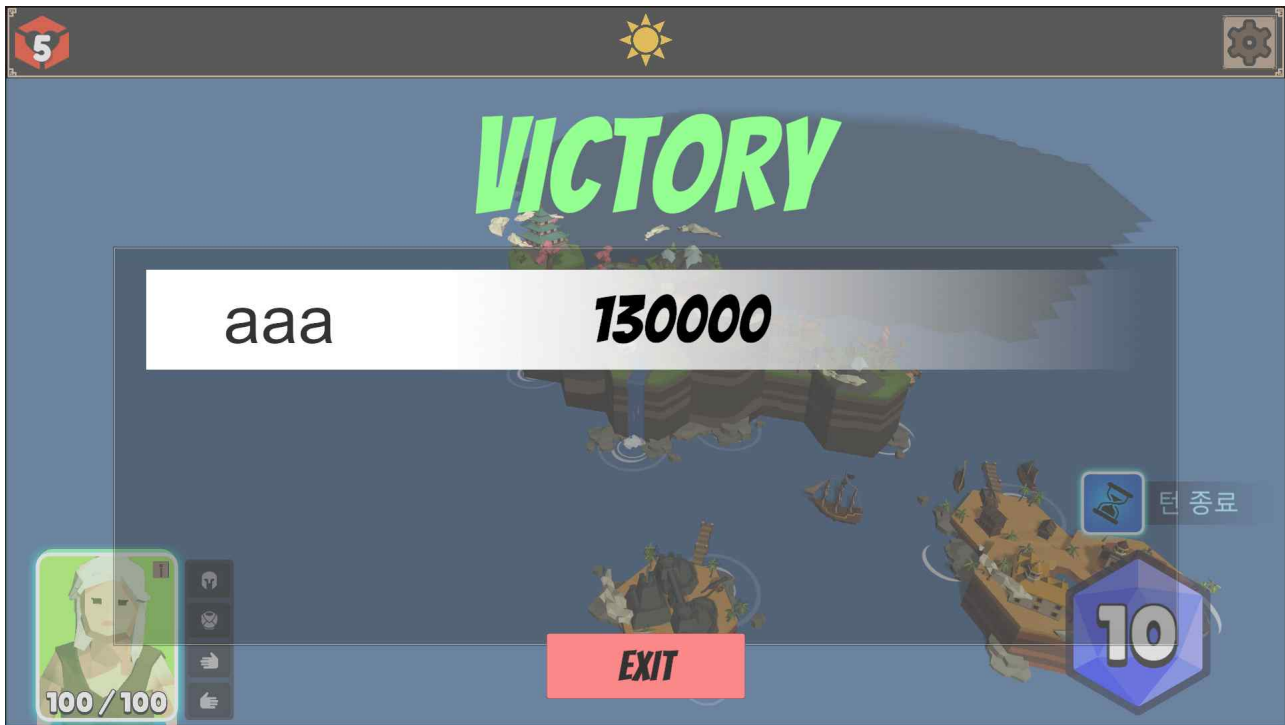
[그림 6-18] - 언어 변경을 위한 설정 창 UI

아래 [그림 6-19]은 적의 스탯 확인을 위한 UI이다. 해당 UI는 적 선택 시 팝업창 형태로 표시되며 레벨 및 체력, 공격력을 확인할 수 있다.



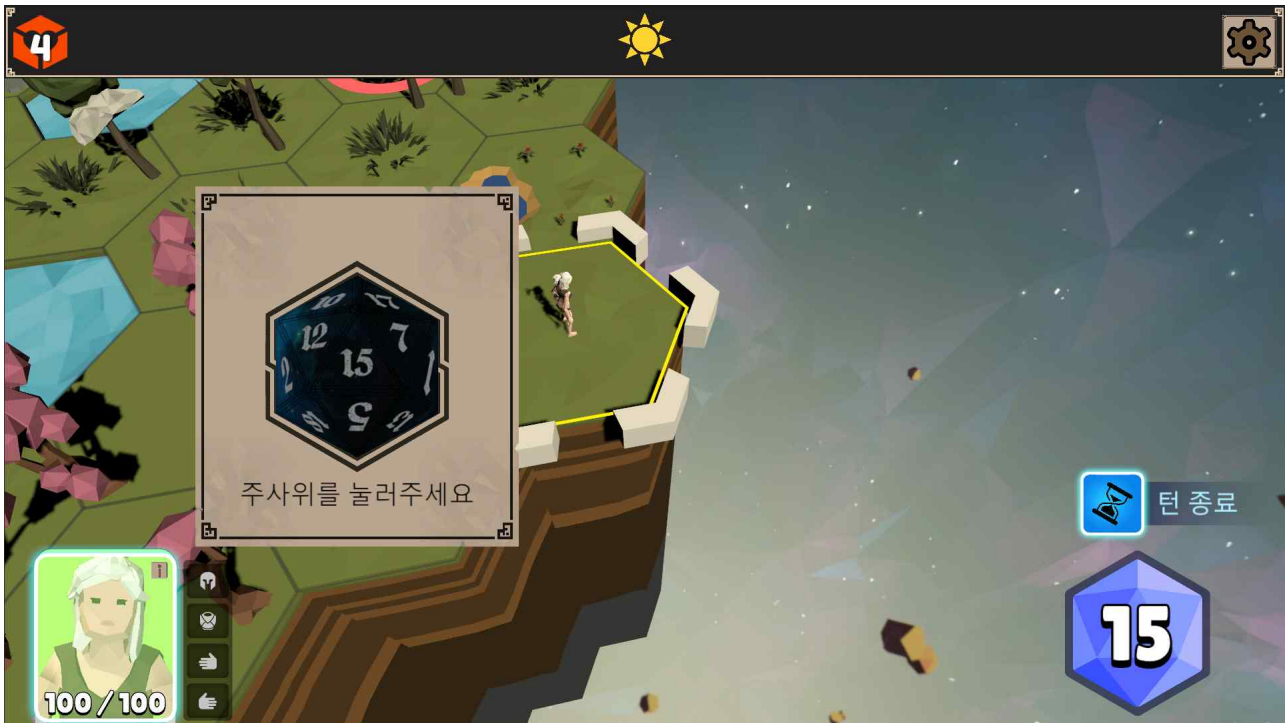
[그림 6-19] - 적 스탯 확인을 위한 UI

아래 [그림 6-20]은 게임 클리어 시 나타나는 게임 클리어 UI이다.



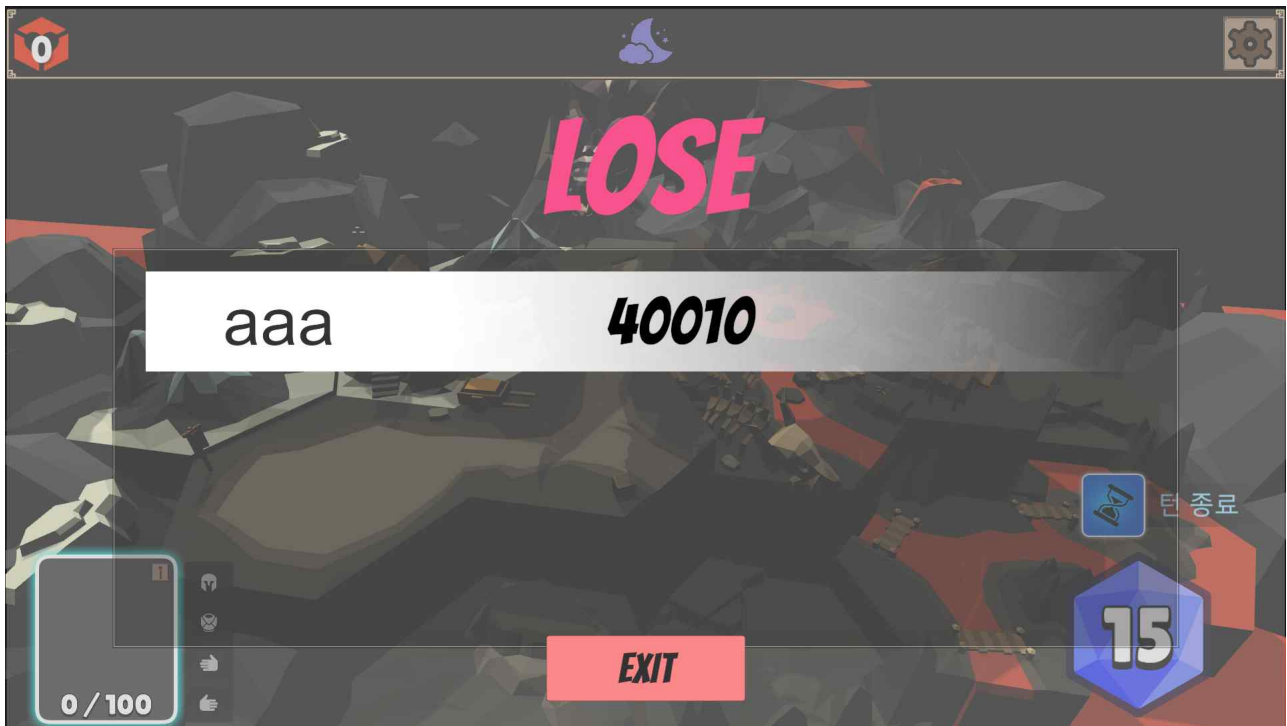
[그림 6-20] - 게임 클리어 UI

만약 게임 플레이 중 사망하게 되는 경우 아래 [그림 6-21]과 같이 좌측 상단의 목숨 수가 하나 줄어들고, 임의의 캐릭터 부활 위치에서 부활하여 다시 주사위 굴리기부터 시작하게 된다.



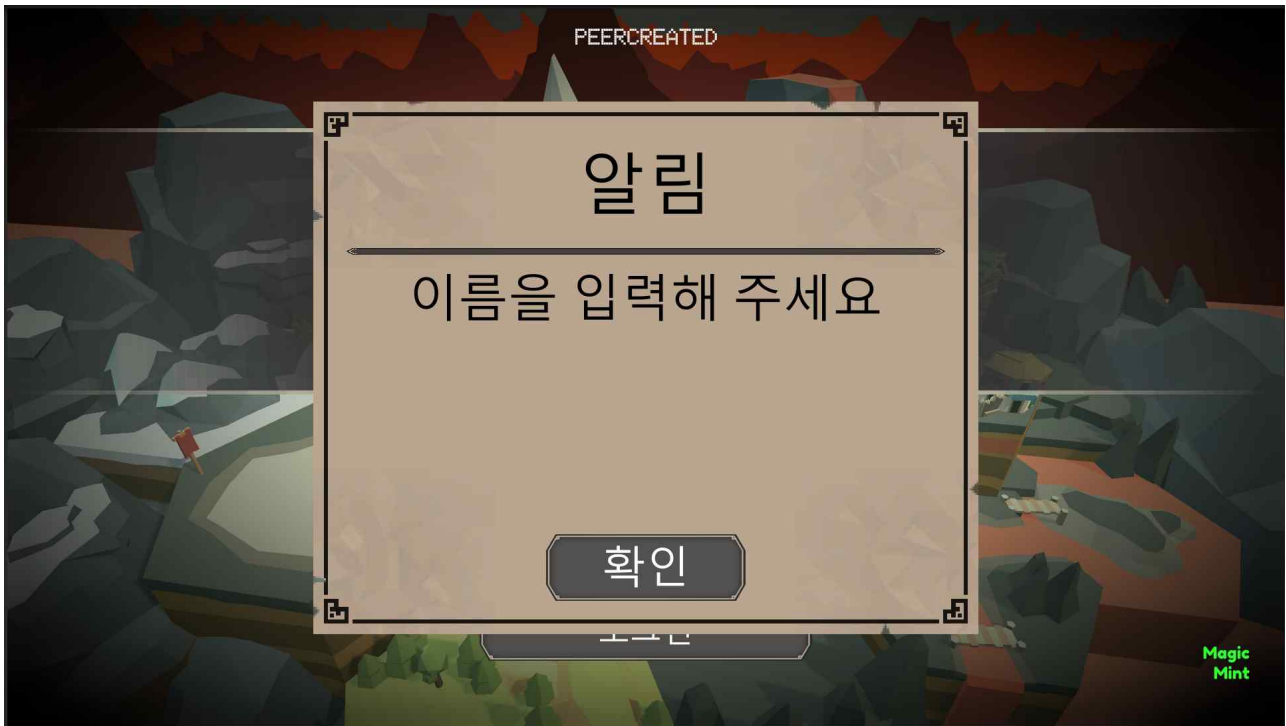
[그림 6-21] - 사망 후 부활

만약 남은 목숨이 없는 상태에서 사망하게 되는 경우 게임 오버가 되어 게임이 종료된다. 아래 [그림 6-22]는 게임 오버 시 나타나는 게임 오버 화면이다.



[그림 6-22] - 게임 오버 UI

또한 게임 시스템 전반에 있어 맨 처음 타이틀 화면에서 이름을 입력하지 않고 게임에 접속하려 하거나, 멀티 플레이 중 게임 방에 있는 플레이어가 전부 레디하지 않았을 경우 등과 같이 알림 메시지를 표시하여 플레이어가 비정상적인 행동을 하는 것을 막는다. 아래 [그림 6-23]은 알림 메시지가 표시되는 알림창 UI로, 해당 알림은 타이틀 화면에서 이름을 입력하지 않고 게임에 접속하려 했을 경우 표시되는 UI이다. 만약 다른 원인으로 인해 알림이 발생한 경우 알림창의 메시지만 변경되어 해당 경우에 맞게 표시된다.



[그림 6-23] - 게임 알림창 UI

7. Implementation requirements

H/W platform requirements

- CPU : intel i3+
- RAM : 4G+
- HDD / SSD : 10G+

S/W platform requirements

- OS : Windows7+
- Implement Language : C#
- Unity 2021.3.30f1+

Server platform requirements

- Photon Server
- Windows Server 2012 R2 x64+
- Microsoft .NET Framework 4.5+
- Development environment : Microsoft Visual Studio 2013+

8. Glossary

이름	설명
Unity	게임 개발을 위한 게임 엔진으로 본 프로젝트의 게임 개발이 진행되는 프로그램이다.
Photon Server	Unity 상에서 멀티 플레이를 지원하기 위해 본 프로젝트에서 채용한 서버 프로그램이다.
싱글톤	생성자가 여러 차례 호출되더라도 실제로 생성되는 객체는 하나로, 최초 생성 이후 호출되는 생성자는 최초의 생성자가 생성한 객체를 리턴하는 소프트웨어 디자인 패턴이다.
XML	W3C에서 개발된, 다른 특수한 목적을 갖는 마크업 언어를 만드는데 사용하도록 권장하는 다목적 마크업 언어이다. 본 프로젝트에서는 게임 시스템 내 데이터 파일 용도로 사용된다.
마크업 언어	태그 등을 이용하여 문서나 데이터의 구조를 명시화하는 언어의 한 가지이다.

9. References

- [1] Alan Dennis, Barbara Haley Wixom and David Tegarden, Systems Analysis and Design with UML: An Object-Oriented Approach, 5th edition, Wiley, 2016.
- [2] Karoly Nyisztor, UML and Object-Oriented Design Foundations, Independently published, 2018.
- [3] Grady Booch, James Rumbaugh, and Ivar Jacobson, Unified Modeling Language User Guide, 2nd Edition, Addison-Wesley Professional, 2005.
- [4] Unity 사용자 매뉴얼 2021.3(LTS)
<https://docs.unity.cn/kr/2021.3/Manual/UnityManual.html>
- [5] Photon 기술 문서 일람
<https://doc.photonengine.com/ko-kr/fusion/current/getting-started/fusion-intro>